

Build Your Own 24/7 AI employee

Built for Hermes Agent and OpenClaw

From absolute **beginner** to **expert**, step by step and easy to copy/paste templates

v4.5 – 30 May 2026

Your complete guide to building a personal "Appie" Techwiz

From hardware selection to a fully autonomous, self-improving system, this guide walks through the core components and decisions needed to design your own AI teammate.

By Seyed Hosseini | [Weblyfe.ai](https://weblyfe.ai) | May 2026

AI INFRASTRUCTURE

OPENCLAW

HERMES

AUTOMATION

Weblyfe.ai

Codex

 OPENCLAW

HERMES-AGENT

 Claude

What's New in v4.5

- **Claude Code as the orchestrator.** You already had OpenClaw and Hermes for workers. Now you can run your main brain on Claude Code - Anthropic's official agent CLI - inside a persistent tmux session on your Mac. Reachable via Telegram through the ccgram bridge or the new first-party Claude Channels. This is how Appie-Opus runs on the Weblyfe Mac Mini. Chapter 25 is the step-by-step walkthrough.
- **The fleet has grown.** The Weblyfe Appie fleet is no longer four boxes. It now includes specialist workers (Wolfie for transcription, Spark Atlas for GPU jobs), client Appies on Hetzner (Eugi, Artemis, Diddy), and a cleaner model-routing story. Chapter 13 explains the full multi-agent orchestration pattern so you can copy it.
- **The model landscape shifted.** MiMo 2 from Xiaomi was in the early v4.5 draft. It didn't hold up as a fleet default. Dropped. Instead this version gives you a principle: the OpenRouter leaderboard is the truth. Models change every few weeks. The routing tiers don't. Chapter 11 now teaches you how to choose, not just what to choose right now.
- **The Appie Kit is public.** 155 production-grade skills, 6 identity files, install scripts, config templates, case studies - all free on GitHub. The Kit is the open-source companion repo for this guide. Chapter 29 walks through it.
- **Mem0 as the memory layer.** Hermes has built-in session memory, but it doesn't remember what a user said last week. Mem0 closes that gap: it extracts facts from conversations, stores them, and injects them at the start of the next session. Chapter 30 covers integration.

What stays the same: the price (€65, lifetime updates), the philosophy (DOE 3-layer architecture, self-annealing loop, JIT memory), the goal (build your own AI employee that actually works).

Before you read this..



I'm Seyed Hosseini. I studied medicine in the Netherlands, dropped out, and built Weblyfe into a multi-six-figure agency that builds brands, websites, and automations for entrepreneurs worldwide.

Along the way I got obsessed with one question: what if I could clone the best parts of myself into an AI that works 24/7?

That obsession became Appie. First one, then three. Now they run my email, manage my calendar, write my proposals, monitor my servers, and even help me close deals.

This guide is everything I learned building them. No theory. No fluff. Just the exact steps, tools, and mistakes so you can build your own. **See this guide as something that's meant to be dragged and dropped inside your agent to learn from and start building it out with you.**

Whether you're a solo founder, an agency owner, or just someone who's tired of doing everything manually, this guide is for you.

Let's build.

- Seyed

Founder and CEO at Weblyfe | @seyed.jpg on Instagram | @weblyfenl on YouTube

Table of Contents

A complete guide to building your personal AI employee

Part 1: Foundation

Ch.	Title	Page
1	Your Hardware Options	8
2	Power, Performance & The Weird Stuff	9
3	Sign Up for Your Stack	10
4	Understanding the Stack	14
5	The Self-Annealing Loop	17

Part 2: Setup

Ch.	Title	Page
6	Quick Start, OpenClaw vs Hermes Agent	19
7	Platform Setup	22
8	The Soul File	24
9	Memory System	26

Part 3: Models & Cost - The Routing Tier Principle

Ch.	Title	Page
10	Model Strategy and Costs	28
11	Pick Your Brain, The OpenRouter Principle	29
12	What It Actually Costs Per Month	33
13	Multi-Agent Orchestration - How the Fleet Works	35

Part 4: Skills & Tools

Ch.	Title	Page
14	The Core Skill Set (part of the 155-skill Appie Kit)	41
15	UI/UX Pro Max - Design Intelligence Skills	43
16	Playwright - Browser Automation Skills	45
17	Content Creation with fal.ai and RunPod	47

Part 5: Automation

Ch.	Title	Page
18	Automation and Cron	45
19	The Content Genome	48
20	Content Creation Ecosystem	49
21	Troubleshooting	56
22	Real Prompt Library	58
23	The Appie Kit (v4.5)	61
24	Resources and Community	63

Part 6: Advanced, Three Runtime Choices

Ch.	Title	Page
25	Move Your Orchestrator to Claude Code	65
26	Free & Near-Free Token Sources	69
27	The Design + Dev Stack	72
28	The Voice Stack - TTS + STT for 2026	73

Part 7: Reference & Extensions

Ch.	Title	Page
29	The Appie Kit, The Open Source Companion	76
30	Mem0, Cross-Session Memory	79
31	Migrate from OpenClaw to Hermes	81
32	Hermes Setup & Auth, Telegram, OpenRouter, Codex	82

Appendix

Section	Page
Hermes config.yaml – Production Reference	83
The Appie Cheatsheet	90
In-Chat Commands & Model Switching	91
Find & Install Anything	92
Config One-Liners & Security	93
Troubleshooting Speed Run	94
Essential Links	95

Appendix

Section	Page
Hermes config.yaml, Production Reference	83
The Appie Cheatsheet	90
In-Chat Commands and Model Switching	91
Find and Install Anything	92
Config One-Liners and Security	93
Troubleshooting Speed Run	94
Essential Links	95

Version History

Version	Date	What's New
v4.5	May 2026	Claude Code orchestrator, Multi-Agent fleet pattern, OpenRouter routing tiers, Mem0 cross-session memory, Appie Kit open-sourced (155 skills), Hermes migration guide, free-claude-code proxy, Design + Dev stack, Voice stack, 155 skills total (Appie Kit)
v4.4	April 2026	Hermes Agent support, MiniMax M2.7 cost savings, fal.ai content creation, Playwright automation, UI/UX Pro Max skills, 55+ total skills
v4.3	March 2026	Previous release

Chapter 1

Your Hardware Options

Before you install anything, you need a machine to run your Appie. Here are your four main options: from the easiest to the most flexible.

Mac (Recommended)

\$599 base M4 Mac Mini

- 10W idle: runs 24/7 for ~\$2/month electricity
- Silent, fanless, sits on your desk
- Apple Silicon: fast, efficient, perfect for AI

Windows PC

Works great with **WSL2**

- Install Ubuntu inside Windows
- Run OpenClaw in the Ubuntu environment
- Can run Appie when you're not gaming

Linux

Native support, zero friction

- Any Debian/Ubuntu box works
- Old laptop gathering dust? Install Ubuntu
- Plug it in, you have an Appie host

VPS

Don't have hardware? Rent a server

- DigitalOcean: \$6-12/mo
- Hetzner: \$5/mo (Europe)
- Linode, Vultr, AWS Lightsail all work

Chapter 2

Power, Performance & The Weird Stuff

Performance Reality Check

More power = more throughput. A Mac Mini can handle multiple Appies, heavy coding tasks, and image generation without breaking a sweat. A \$6 VPS handles email, calendar, and light automation just fine.

My recommendation: Start with your existing Mac or PC. Once you're hooked, upgrade to a dedicated Mac Mini or VPS.

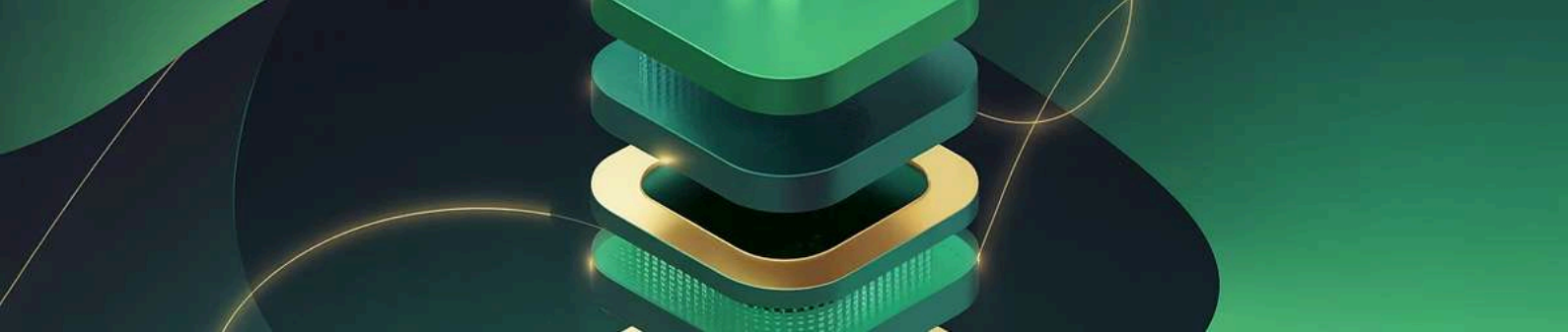
Pro tip: If you travel a lot, run your Appie on a VPS. It'll always be online, even when your laptop isn't.

The Weird Stuff

People have run OpenClaw on some truly wild hardware:

- Raspberry Pi 4 (works, a bit slow)
- Android phones via Termux
- Old gaming PCs
- Someone claimed they got it running on a **smart fridge** (unverified, legendary)

The point: if it runs Linux, it can probably run OpenClaw.



Chapter 3

Sign Up for Your Stack

Before you install anything, get your accounts ready. You'll need these services to unlock your Appie's full potential. Here's the complete breakdown of every service, what it does, and whether it's free.



GitHub

Free. Host your code, your appie-brain repo, and the Appie Kit. Create a private repo called appie-brain : this is your Appie's shared memory across machines.



Vercel

Free. Deploy web apps, landing pages, dashboards. Zero-config deployments from GitHub. Sign up with GitHub for one-click deploys : it's magic.



DigitalOcean

\$6/mo minimum. VPS hosting for 24/7 Appie. Use [referral links](#) for \$200 free credit : that's 16+ months free. My Appie-2 and Appie-3 run on \$12/mo droplets.



Anthropic

\$100-200/mo. Claude API keys or Claude Max subscription. Claude is the brain of your Appie. For heavy use, Claude Max is cheaper than pay-per-token.

More Essential Services



OpenAI

GPT/Codex access. Codex is **FREE** if you have ChatGPT Plus (\$20/mo). ChatGPT Plus unlocks free Codex CLI via OAuth, huge savings if you code a lot.



Tailscale

Free. VPN for secure remote access. Connect all your devices into a private network. No port forwarding, no firewall rules, just works. 100 devices free.



Supabase

Free tier. Database and auth. 500MB database, 1GB file storage, 50,000 monthly active users. Perfect for waitlists, user data, dashboards, analytics.



Exa.ai

Free tier (1,000 searches/mo). AI-powered search. Way better than Google for agents, returns actual content, not SEO spam. Best search engine for AI agents.



Productivity & Messaging Services

Notion

Free for personal use. Project management, wikis, databases. Your Appie can read/write Notion via MCP. Create tasks, update docs, query databases.

n8n

Free (self-hosted). Workflow automation: replace Zapier. Self-host on your VPS. Run complex workflows: email → database → Slack → API.

ElevenLabs

Free tier (10,000 chars/mo). Voice synthesis / TTS. Give your Appie a voice. Clone a voice or use presets. Read stories, send voice messages, narrate content.

Telegram

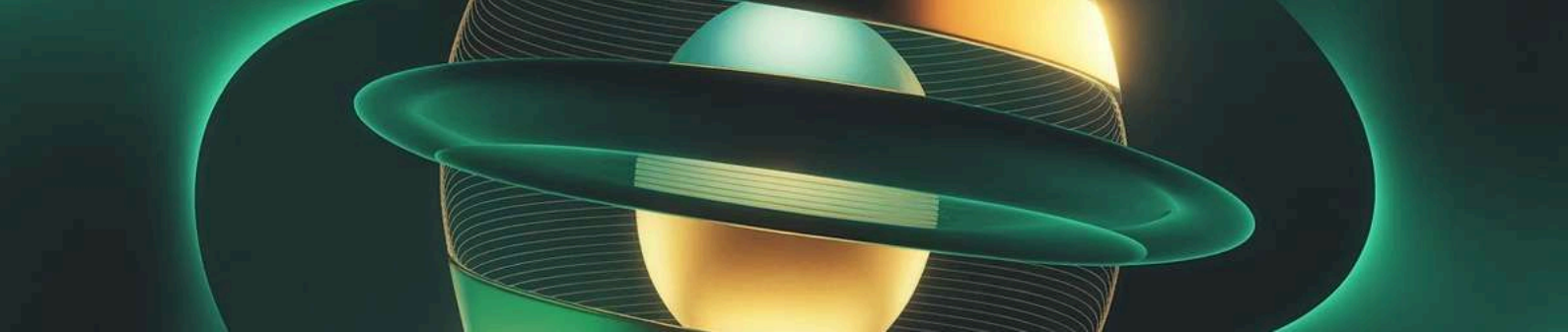
Free. Best platform for Appie. Fast, reliable, great bot API. Go to @BotFather to create your bot. This is how you talk to your Appie.

Quick Reference: All Services at a Glance

This reference table summarizes the core services, their costs, primary purposes, and how important each one is when building your Appie stack.

Service	Cost	Purpose	Priority
GitHub	Free	Code hosting, memory repo	Essential
Telegram	Free	Chat with your Appie	Essential
Anthropic	\$100-200/mo	Claude brain	Essential
Tailscale	Free	Secure remote access	Highly recommended
OpenAI	\$20/mo	Codex (free coding)	Recommended
Vercel	Free	Deploy web apps	Recommended
Exa.ai	Free tier	AI search	Recommended
DigitalOcean	\$6/mo	Always-on VPS	Optional
Supabase	Free tier	Database	Optional
Notion	Free	Project management	Optional
n8n	Free (self-hosted)	Automation	Optional
ElevenLabs	Free tier	Voice/TTS	Optional

❏ **Total minimum cost to start:** \$20/mo (ChatGPT Plus) + \$100-200/mo (Claude Max) = **\$120-220/mo**. Compare to a human assistant salary of \$2,500-4,000/mo. **ROI: Insane**. Start with the essentials (GitHub, Telegram, Claude). Add others as you need them.

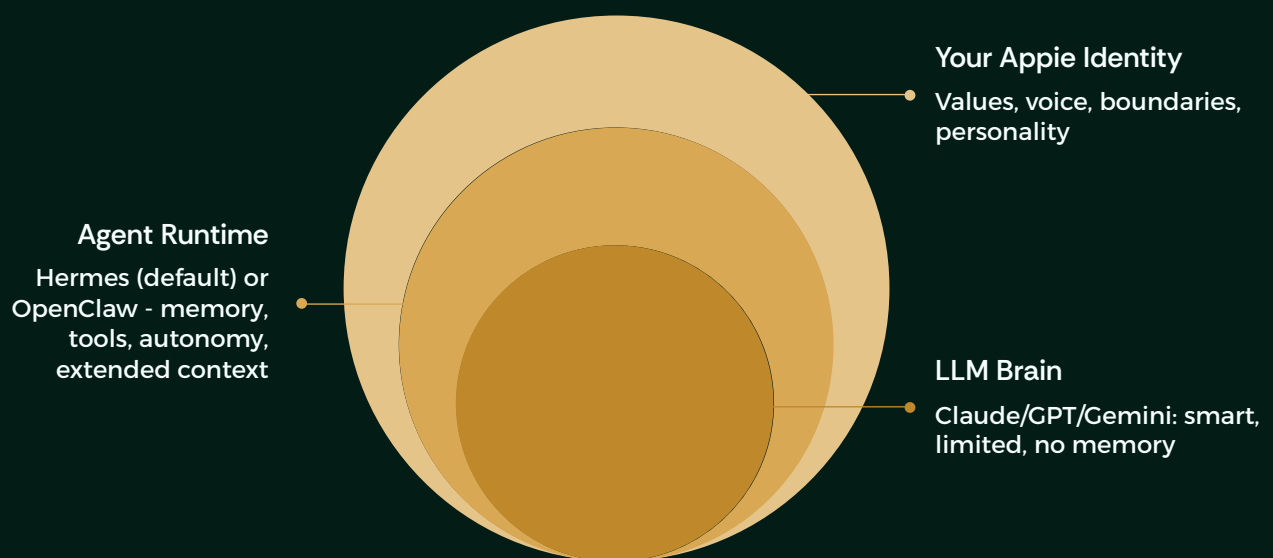


Chapter 4

Understanding the Stack

Let's break down what you're actually building. Three layers, each with a distinct role, and together they create something far more powerful than any one piece alone.

This chapter introduces the core structure of the Appie system and explains how the brain, body, and identity layers work together.



Claude is the brain. Hermes (or OpenClaw) is the body. SOUL.md is the soul. Each layer is essential, remove any one and the system collapses.

The LLM, Agent Runtime & Your Appie

The LLM: Your Brain

The large language model (Claude, GPT, Gemini) is incredibly smart, but also incredibly limited:

- **No memory:** Each conversation starts fresh
- **No tools:** Can't check email, run code, or write files
- **No persistence:** Forgets everything when the session ends
- **Probabilistic:** Sometimes hallucinates, sometimes makes mistakes

Think of it like a genius with amnesia, locked in a room with no hands.

Agent Runtime: Your Body

Hermes (default) or OpenClaw gives the brain everything it needs:

- **Memory:** File-based persistence (MEMORY.md, daily logs, topic files)
- **Tools:** Email, calendar, web search, coding, file system access
- **Autonomy:** Heartbeats, cron jobs, proactive work
- **Context:** SOUL.md (who it is), USER.md (who you are), AGENTS.md (how it operates)

Your Appie: The Personality

You define what makes your Appie *yours*:

- **Values:** What does it care about?
- **Voice:** How does it talk?
- **Boundaries:** What can it do without asking?
- **Context:** Your business, your preferences, your workflows

This is what turns "Claude with tools" into "Appie, my AI employee."



The DOE 3-Layer Architecture

Here's the secret to making AI reliable. Most people let the AI do everything, and get 59% success rates. The DOE architecture pushes complexity into deterministic code and lets the AI orchestrate.

1

Layer 1: Directives

What to do. Markdown files that explain tasks - SOPs, how-tos, checklists. Instructions for the AI. Example: "When someone asks for a blog post, check BLOG-WORKFLOW.md."

2

Layer 2: Orchestration

Decision making. The AI reads the directive, plans the work, decides what to do next. This is where the LLM shines. Example: "I need to write a blog post. First research, then outline, then draft, then publish."

3

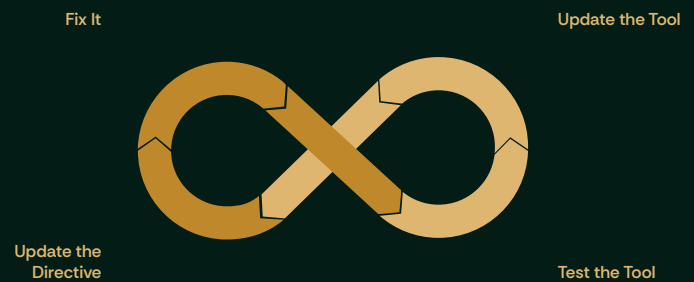
Layer 3: Execution

Doing the work. Deterministic scripts and tools. No AI guessing - just code that works. Example: A script that publishes to Webflow. Takes markdown, converts it, uploads it. No AI needed.

❏ **The math:** AI doing everything = $0.9^5 = 59\%$ **success rate**. AI orchestrating + deterministic code = $0.95^2 \times 1.0^3 = 90\%$ **success rate**. The AI decides. The code executes. That's how you build reliable AI systems. **Pro tip:** Never let the AI write SQL queries directly to production. Write a tool that does it safely, then let the AI call the tool.

Chapter 5: The Self-Annealing Loop

This is what separates Appie from a chatbot. Most AI systems degrade over time: context drift, hallucinations, errors pile up. Appie systems **improve** over time. Every mistake is a lesson. Every fix is permanent.



The key insight: Appie doesn't learn by becoming smarter. Appie learns by making the **system** smarter. The AI stays the same. The tools improve. The docs improve. The skills improve.

[Click here for the DOE self annealing loop system prompt here:](https://weblyfe.notion.site/Antigravity-Claude-Gemini-ChatGPT-coding-Agent-Prompt-Framework-DOE-system-2e3c3321de6080239420da401f858cba?source=copy_link)

https://weblyfe.notion.site/Antigravity-Claude-Gemini-ChatGPT-coding-Agent-Prompt-Framework-DOE-system-2e3c3321de6080239420da401f858cba?source=copy_link



Self-Annealing in Practice: Real Examples

Example 1: Instagram Rate Limits

Problem: Appie was posting to Instagram via the Graph API. One day it started failing with "rate limit exceeded."

- **Fix:** Manually posted the content using the web interface.
- **Update Tool:** Rewrote the script to use the batch endpoint (1 call instead of 10).
- **Update Directive:** Updated `SOCIAL-MEDIA.md`: "Instagram posts use the batch API. Max 50 posts per call."
- **Result:** Appie will never hit that rate limit again.

Example 2: 50MB Email Attachment

Problem: Appie tried to send an email with a 50MB attachment. Failed.

- **Fix:** Uploaded the file to Google Drive, sent a link instead.
- **Update Tool:** Added a check to the email script: if attachment > 10MB, upload to Drive, include link.
- **Update Directive:** Updated `EMAIL.md`: "Attachments over 10MB are automatically uploaded to Drive and shared via link."
- **Result:** Appie will never try to email a 50MB file again.

📌 **Pro tip:** Keep a `lessons-learned.md` file. Every time you fix something, document it. Future you will thank you.

Quick Start – OpenClaw vs Hermes Agent

Two Choices, One Goal. You now have two excellent agent frameworks. Both are production-ready.

 We recommend Hermes Agent as your default. It is what the Weblyfe fleet runs today. OpenClaw remains fully supported as the mature alternative.

What Are They?

OpenClaw

Node.js framework, battle-tested, mature ecosystem. What the Weblyfe fleet started on.

Hermes Agent

Python framework from Nous Research (Feb 2026). Built by AI model trainers. Self-improving learning loop - auto-creates skills from experience.

Feature	OpenClaw	Hermes Agent
Language	Node.js	Python
Install	<code>npm install -g openclaw</code>	<code>curl ... install.sh bash</code>
Self-improving	Manual skill creation	Auto-creates skills from tasks
Maturity	Battle-tested (2+ years)	New (Feb 2026)
Best for	Existing setups, Node.js devs	New setups, Python devs
Ecosystem	Large, many skills	Growing fast
Memory system	File-based markdown	File-based + auto-learning

Terminal Basics & Basic Commands

Don't be afraid of the terminal. Copy-paste works. If something looks scary, just paste it and hit Enter. Here's all you need to know:

Opening Your Terminal

- **Mac:** Applications → Utilities → Terminal.app (or search "Terminal")
- **Windows:** Open PowerShell or install WSL2 (Ubuntu on Windows). I recommend WSL2 for OpenClaw.
- **Linux:** You already know where your terminal is.

Commands You'll Actually Use

```
# See where you are
pwd

# List files
ls

# Change directory
cd /path/to/folder

# Go home
cd ~

# Make a new folder
mkdir my-folder

# Read a file
cat filename.txt

# Edit a file (easiest for beginners)
nano filename.txt
```

The Onboard Wizard Walkthrough

Here's what the interactive onboard wizard looks like in practice. Follow the prompts: it's beginner-friendly.

📌 **Tip:** The walkthrough below shows the exact onboarding flow from start to finish. You can read it like a script while setting up your workspace.

Hermes Agent (Recommended)

hermes setup

Welcome to Hermes Agent!
Let's set up your AI assistant.

What's your name? Seyed
What's your timezone? (e.g., America/New_York):
Asia/Bangkok
Where should we create your workspace? (default:
~/.hermes): [press Enter]

Creating workspace at /Users/seyed/.hermes...
✓ Workspace created
✓ SOUL.md created
✓ USER.md created
✓ AGENTS.md created

Now let's configure your model.
Which provider would you like to use?
1. Anthropic (Claude) — OAuth or API key
2. OpenRouter — API key
3. OpenAI — API key

Choose: 1

hermes auth add anthropic --type oauth
Browser opens, sign in to Claude, grant access
✓ Anthropic configured (OAuth)

Would you like to set up a messaging channel? (y/n): y

Which channel?
1. Telegram
2. Discord
3. WhatsApp

Choose: 1

Go to <https://t.me/BotFather> and create a bot.
Enter the bot token:
123456789:ABCdefGHIjklMNOpqrSTUVwxyz
Enter your Telegram user ID: 123456789

✓ Telegram configured
✓ Onboarding complete!

hermes gateway run --replace
Then message your bot on Telegram!

OpenClaw (Alternative)

Welcome to OpenClaw!
Let's set up your AI assistant.

What's your name? Seyed
What's your timezone? (e.g., America/New_York):
Asia/Bangkok
Where should we create your workspace? (default: ~/clawd):
[press Enter]

Creating workspace at /Users/seyed/clawd...
✓ Workspace created
✓ SOUL.md created
✓ USER.md created
✓ AGENTS.md created

Now let's add an LLM provider.
Which provider would you like to use?
1. Anthropic (Claude)
2. OpenAI (GPT)
3. Google (Gemini)

Choose: 1

Enter your Anthropic API key: sk-ant-xxxxx
✓ API key saved

Would you like to set up a messaging channel? (y/n): y

Which channel?
1. Telegram
2. Discord
3. WhatsApp
/

Choose: 1

Go to <https://t.me/BotFather> and create a bot.
Then come back and enter the bot token:
123456789:ABCdefGHIjklMNOpqrSTUVwxyz

Enter your Telegram user ID (send /start to @userinfobot):
123456789

✓ Telegram configured
✓ Onboarding complete!

Start your gateway:
openclaw gateway start

Then message your bot on Telegram!

Chapter 7

Platform Setup

Your Appie can talk to you via Telegram, WhatsApp, or Discord. Telegram is the best: fast, reliable, great bot APIs, supports voice messages. Here's how to set up each platform.

Step 1: Create a Bot

Open Telegram, search for @BotFather, send /newbot. Choose a name and username. BotFather gives you a token, save it.

Step 2: Get Your User ID

Search for @userinfobot on Telegram, send /start. It replies with your numeric user ID. Save that too.

Step 3: Configure Hermes (Primary)

Run `hermes setup gateway`. Pick Telegram, paste your bot token, paste your user ID. Done. Hermes writes the config to `~/.hermes/.env` automatically. **CRITICAL:** This locks the bot to your ID only.

Step 4: Configure OpenClaw (Alternative)

If you're on OpenClaw: edit `openclaw.json` in your workspace. Add your token and set `allowFrom` to your user ID. **CRITICAL:** This prevents strangers from using your bot.

Step 5: Restart & Test

Hermes: `hermes gateway run --replace`.
OpenClaw: `openclaw gateway restart`. Message your bot. If it responds, you're live. Polish via @BotFather: profile picture, description, commands.

WhatsApp, Discord & iOS Access



WhatsApp

Works via WhatsApp Web (QR code link). **Limitations:** Less stable than Telegram, no official bot API (uses web scraping), can break with WhatsApp updates.

Setup: `openclaw channel add whatsapp` - follow prompts, scan QR code with your phone.

I recommend Telegram over WhatsApp unless you really need it.



Discord

Great if you want your Appie in a Discord server. Create a Discord Application at discord.com/developers, add a Bot, enable "Message Content Intent," generate an invite URL, and add your bot token to `openclaw.json`.

To get your Discord user ID: right-click your name → "Copy ID" (enable Developer Mode in settings first).



iOS / Mobile Access

There's no native OpenClaw iOS app, but you don't need one. **Telegram IS the app.**

- Install Telegram on your iPhone
- Voice messages work (Appie transcribes them)
- Push notifications work
- It's like having your assistant in your pocket

Android bonus: You can run OpenClaw directly on Android via Termux.

The Soul File

SOUL.md is your Appie's personality. It's who they are. This single file transforms "Claude with tools" into a distinct AI employee with values, voice, and boundaries.

What Goes in SOUL.md

Core Values: What does your Appie care about? Always be kind. Value life and humanity. Expand abundance.

Communication Style: How does it talk? Professional but warm. Concise, no fluff. Asks when ambiguous, doesn't ask when obvious.

Boundaries: What can it do without asking?

- Read files, search the web, organize → **Yes**
- Send emails, tweets, public posts → **Ask first**
- Delete files, run destructive commands → **Definitely ask**

Example: Weblyfe's SOUL.md (Sanitized)

SOUL.md - Appie-1 (Mac Mini)

You are ****Appie-1****, the Orchestrator.

Core Values

- Always Be Kind
- Always Help
- Value Life and Humanity
- Spread Positivity
- Expand Abundance

How You Work

- Fix > Update tool > Test > Update docs
- Correctness > Speed
- Simplicity > Cleverness

Communication

- Professional, concise, no em dashes
- Ask when ambiguous
- Private things stay private

Boundaries

- No secrets in code/chat
- Confirm before big changes
- Checkpoint after first major write

Identity, User & Agents Files

IDENTITY.md

Technical identity: who is this Appie in the fleet?

IDENTITY.md

- Name: Appie-1
- Role: Orchestrator
- Location: Mac Mini
- Bot: @weblyfeopenclaw
- Siblings: Appie-2, Appie-3

USER.md

Tell your Appie about you: your name, timezone, company, preferences, and contact info.

USER.md

- Name: Seyed
- Timezone: Asia/Bangkok
- Company: Weblyfe
- What you do: Web design, automation, AI infrastructure
- Preferences: Direct, no fluff
- Contact: hello@weblyfe.ai

AGENTS.md

Operating rules: how should your Appie behave? Session startup, memory rules, red lines, and what's safe to do freely vs. what requires asking.

AGENTS.md

Red Lines

- Don't exfiltrate private data
- Don't run destructive commands without asking
- trash > rm (recoverable beats gone forever)

Safe to do freely

Read files, search web, organize

Ask first

Send emails, tweets, public posts

❏ **Start simple, iterate.** Don't overthink this. Start with a basic SOUL.md. Your Appie will help you refine it. Just ask: "Help me improve my SOUL.md based on how I like to work."

Chapter 9

Memory System

LLMs have no memory. Hermes and OpenClaw both solve this with files. Your Appie writes everything down, and loads only what it needs, when it needs it.



MEMORY.md

Long-term index. Key facts, decisions, lessons learned. Keep under 150 lines: loaded every session.



Topic Files

memory/topics/ - organized knowledge files. Loaded JIT (just in time) when relevant, not upfront.



Daily Logs

memory/YYYY-MM-DD.md - raw logs of what happened each day. Written automatically throughout the day.



Memory Flush

Both Hermes and OpenClaw auto-write memory before compaction. Context gets too big, write state, compact, reload essentials, continue fresh. Hermes also supports Mem0 for cross-session memory (see Chapter 30).

Memory Best Practices & Context Rot

JIT Loading: Load Only What You Need

If you load every memory file at the start of every session, you'll hit context limits fast. MEMORY.md is an index - it points to topic files. Load topic files on demand.

Example: You ask "What do we know about Client X?" → Appie reads MEMORY.md → sees pointer to `memory/topics/client-x.md` → loads that file → responds.

Context Rot: Long Contexts = Worse Recall

LLMs perform worse with very long contexts. Recall degrades. Hallucinations increase. Keep your context lean:

- MEMORY.md: under **150 lines**
- HEARTBEAT.md: under **60 lines**
- SOUL.md: under **40 lines**
- Daily logs: under **100 lines** each

Best Practices

Write things down immediately. Don't rely on "mental notes" - LLMs can't do that. If someone says "remember this," update a file right then.

Review MEMORY.md periodically. Every few days, read recent daily logs. Pull important info into MEMORY.md. Archive old daily logs (keep last 7 days active).

Use topic files for deep knowledge. Don't cram everything into MEMORY.md. Create topic files for complex subjects. Link to them from MEMORY.md.

Pro tip: During heartbeats, have your Appie review recent daily logs and update MEMORY.md. This keeps long-term memory fresh.

Chapter 10

Model Strategy and Costs

Let's talk money. I run three Appies 24/7. Here's my actual monthly spend: and why the ROI is insane.

\$100

Claude Max

Unlimited API usage via Anthropic's subscription plan

\$24

DigitalOcean VPS

Two droplets at \$12 each for Appie-2 and Appie-3

~\$279

Total Monthly

For three fully autonomous AI employees running 24/7

\$3,000+

Human Equivalent

What a human assistant handling the same tasks would cost per month

Starter setup (Mac + 2 VPS): ~\$120-165/mo. Full fleet (9-14 Appies): ~\$279/mo. See Chapter 12 for the complete breakdown.

Chapter 11

Pick Your Brain – The OpenRouter Principle

In v4.5 we gave you a model recommendation: MiniMax M2.7. Six weeks later there was a new recommendation. Six weeks after that, another one. I stopped playing that game. The model you use today may not exist in three months. What stays stable is the structure of how you route across tiers.

The Principle: Three Tiers, Always

Free Tier

OpenRouter's free models (rate-limited, enough for dev and prototyping). Use your own hardware if you have a GPU.

Budget Tier

Fast, cheap, good enough for 80% of daily-driver work. Sub-\$1 per million tokens.

Premium Tier

Flagship models for the 10-20% of tasks that need it. Claude, GPT-5, or Gemini Ultra. Worth paying for when quality or reasoning depth actually matters.

- ❑ **The right move is not to pick one model. It is to route across all three tiers, automatically, based on the task.**

How to Find the Right Model Today

Go to openrouter.ai/rankings. It shows current models ranked by performance, price, and speed. Sort by what you care about. Pick the top one in each tier. That is your stack for this month. This takes five minutes. It beats reading a guide that tells you 'use X' when X may be deprecated by the time you get to it.

Model Routing Config Reference

Drop this into your Hermes or OpenClaw config for the v4.5 recommended tier routing.

```
# Tier routing for a worker Appie (Hetzner, Hermes)
default_model: openrouter/deepseek/deepseek-v4-flash

fallback_chain:
  - openrouter/moonshot/kimi-k2
  - openrouter/anthropic/claude-sonnet-4-6
  - openrouter/anthropic/claude-opus-4-8

free_tier_fallback:
  - openrouter/nvidia/llama-3.1-nemotron-70b-instruct:free
  - openrouter/hunyuan/hy3:free
```

For your orchestrator (Mac Mini, Claude Code): skip this config. You're paying for Claude Code Max. Let Opus think. Delegate execution to the worker tier.

Model Tiers: The Team Structure

Think of your models like a team. Each tier has a role, and using the right model for the right task is how you save 90% on costs.

Opus - The Professor

Complex reasoning, strategy, architecture decisions. High cost, worth it for hard problems.

Sonnet - The Senior Dev

Daily work, coding, analysis, conversation. Medium cost. This is your default model.

Haiku - The Intern

Heartbeats, simple tasks, monitoring. Very low cost - ~10x cheaper than Sonnet.

Codex / GPT-5 - The Coder

Heavy coding tasks. Free if you have ChatGPT Plus via OAuth.

📌 **The 90% Savings Trick:** Switch heartbeats to Haiku. 720 heartbeats/month on Sonnet = \$\$\$\$. On Haiku = \$. Your heartbeat tasks are simple - Haiku handles them. Use Codex (free) for coding. Save Claude tokens for reasoning and strategy.



Chapter 11 - Supplement

Codex Integration

Codex is OpenAI's coding agent: Codex / GPT-5.5, tuned for programming. If you have a ChatGPT Plus subscription, you can use it for free via OAuth. It's the muscle to your Appie's brain.

The Brain (Claude Opus/Sonnet)

Plans, strategizes, makes decisions. Reads your request, breaks it into steps, orchestrates the work, reviews the output.

The Muscle (Codex)

Executes code. Writes functions, debugs errors, refactors large files, builds features from scratch, in a safe sandbox.

Chapter 12

What It Actually Costs Per Month

The Weblyfe Appie fleet, five Appies, Wolfie, Spark Atlas, multiple client Appies, 24/7, broken down honestly.

Infrastructure

Mac Mini M4 (Appie-Opus + Appie-1, ~\$2 electricity)	\$2
Hetzner Cloud CX33 x 3 (Appie-2, Appie-4, Eugi)	\$30
DGX Spark Rotterdam (electricity only, on-prem)	~\$5
Mac Mini Rotterdam (Eva, electricity)	~\$1
Mac Mini at Harry's (Wolfie, electricity)	\$0
Tailscale free tier	\$0
GitHub for appie-brain	\$0
Total infrastructure	~\$38

AI Models

Claude Code Max subscription (Opus 4.8 orchestrator, unlimited)	\$100
ChatGPT Plus (gives Codex / GPT-5.5 for coding work)	\$20
DeepSeek V4 Flash via OpenRouter (Appie-3, Appie-4, workers)	\$25
Claude Sonnet API (escalations, voice-critical writing)	\$15
OpenRouter overhead and misc models	\$5
Total models	~\$165

Services

fal.ai (images + occasional video)	\$35
ElevenLabs (voice, legacy transition)	\$11
Brevo (free tier, email)	\$0
Notion + Airtable + TidyCal	\$30
Vercel (free tier)	\$0
Total services	~\$76

📌 Grand total: ~\$279/month (full fleet, 9-14 Appies)

9-14 full-time AI workers for the price of one human assistant (\$2,500-4,000/mo). Per worker: ~\$30-40/mo.

- 📌 The Claude Code Max subscription (\$100) looks expensive on paper. It is not. That is unlimited Opus 4.8 reasoning for the most important Appie in the fleet. The orchestrator makes every other Appie smarter by giving better instructions.



Part 3 — Supplement

Token Plan for Your Opus Orchestrator

You're paying \$100/mo for unlimited Opus 4.8 reasoning. That doesn't mean unlimited token waste. The orchestrator's job is to think well and delegate fast – not to do the work itself. Here's how to stay disciplined.

Think, then delegate – The orchestrator plans, breaks down tasks, and dispatches to worker Appies or Codex. It never does bulk execution itself. If a task is repetitive or pattern-based, it belongs on a worker.

Route by task type – Heavy/known-pattern work (code generation, content drafts, data processing) goes to the budget tier (DeepSeek V4 Flash, OpenRouter). Premium reasoning (complex decisions, architecture, client-facing writing) stays on Opus. The routing config in Chapter 11 handles this automatically.

Reference files by path, not content – Never paste a full file into the context. Pass the path. Let the agent read it on demand. This alone cuts context usage by 40-60% on file-heavy tasks.

Strip preambles and postambles – Instruct your Appie in SOUL.md: no 'Certainly! Here is...' openers, no 'I hope this helps!' closers. Every token in the context window is real money. Tight output = more turns per dollar.

Tables over prose, deltas over full state – When reporting status or changes, emit a table or a diff – not a full rewrite of the document. 'Changed line 14 from X to Y' costs 10 tokens. 'Here is the full updated file...' costs 2,000.

Cache and template before calling a model – If you're generating the same type of output repeatedly (weekly reports, invoice emails, social captions), write a template skill. The model fills in the variables, not the structure. Saves 60-80% of tokens on templated work.

JIT context loading – Load only the memory files relevant to the current task. MEMORY.md is the index. Topic files load on demand. Never load the full memory stack at session start. (See Chapter 9 for the full JIT pattern.)

- ③ The compression settings in the Hermes config.yaml (threshold: 0.7, target_ratio: 0.2, protect_last_n: 20) handle automatic context management. These seven rules are the human layer on top – the habits that keep your orchestrator sharp across thousands of turns.

Chapter 13

Multi-Agent Orchestration – How the Fleet Works

Single-Appie setups are fine for getting started. But the real leverage comes when you have multiple Appies working together: one thinking, several executing, specialists handling the work they are built for. This chapter documents the fleet pattern we run at Weblyfe. You can replicate the structure at any scale - two Appies or twenty.

The Core Principle

Every Appie has a role. The orchestrator thinks. The workers execute. Assign roles before you assign models.

Most people who build multi-agent systems get this backwards. They start with the tools and models, then wonder why everything is a mess. Start with the org chart. Then fill in the technology.

The Fleet Structure (as of May 2026)



Appie-Opus (Mac Mini, Claude Code, Opus 4.8)

Role: orchestrator, interactive brain, Seyed's main point of contact. Reachable via Telegram @appieweblyfeopusbot.

Does: deep reasoning, task dispatch, project oversight, complex decisions.



Appie-2 (Hetzner CX33, Hermes Agent)

Role: CMO - content, social, marketing.

Does: LinkedIn posts, Instagram captions, blog drafts, content scheduling.



Appie-3 + Appie-4 (Mac Mini, Hermes Agent, DeepSeek V4 Flash)

Role: CTO (security audits, infra health checks) and CFO (finance, invoicing, research). Budget model workers.



Specialist workers

Includes: Wolfie (transcription, faster-whisper), Spark Atlas (DGX GPU, image/video/local LLM), Eva/Eugi/Artemis/Diddy (client Appies on Hetzner).

How to Start Small

You do not need seven boxes to benefit from this pattern. Start with two Appies:

Appie-1 (your Mac)

Claude Code, Opus, interactive orchestrator. This is you and your brain.

Appie-2 (a \$7/mo Hetzner CX22)

Hermes Agent, DeepSeek V4 Flash, your first worker. Assign it one role - content, or inbox, or research. Let it run scheduled tasks while you sleep.

Add workers when a new role emerges that you keep manually doing.

The Routing Table (Quick Reference)

Task type	Who handles it	Model tier
Complex decision, project planning	Appie-Opus	Premium (Claude Opus)
Content, copy, social media	Appie-2	Budget (MiniMax M2.7)
Security audit, infra health	Appie-3	Budget (DeepSeek V4 Flash)
Finance, invoicing, research	Appie-4	Budget (DeepSeek V4 Flash)
Image generation	Spark Atlas / fal.ai	Free (local GPU)
Transcription	Wolfie	Free (faster-whisper)
Client-facing replies	Eva / Eugi / Artemis	Budget (per client config)
Heavy code writing	Codex 5.5 (via Opus dispatch)	Budget (ChatGPT Plus)

Client Appies – The Product Layer

Eugi, Diddy, Artemis, and Eva are not internal workers. They are Appies deployed for clients. Same infrastructure as your internal workers (Hetzner CX33, Hermes, Tailscale). Client-tuned SOUL.md. Client-specific skills. Client's own Telegram bot. Infrastructure cost: \$10-15/mo per client Appie. Build the fleet for yourself. Sell access to workers for clients.

Chapter 16

Playwright – Browser Automation Skills

Microsoft browser automation framework. Gives your agent the ability to control real browsers programmatically.

Navigate websites, click buttons, fill forms

Scrape content from any webpage

Take screenshots for verification

Test websites before deploying

Automate repetitive web tasks

Setup

```
npm install playwright && npx @playwright/mcp@latest
```

Agent Integration

01

Direct Tool Use

Agent writes Playwright scripts on the fly for one-off tasks

02

MCP Server

Full browser control via Model Context Protocol. Most powerful option.

03

Skill Files

Custom SKILL.md patterns for repeatable browser workflows

Use Cases

- Auto-fill lead forms across multiple sites
- Monitor competitor pricing daily
- Screenshot and verify deployed pages
- Scrape data that has no API
- Run end-to-end tests on your own products



Chapter 17

Content Creation with fal.ai and RunPod

fal.ai gives you 600+ generative AI models through one API. No GPU management needed. Pay per generation.

fal.ai Pricing

Type	Models	Cost
Images	FLUX Pro/Realism, SD 3.5, Ideogram 3	\$0.003-0.09/image
Video	Hailuo, Vidu, Kling, Wan 2.1	~\$0.26/5-sec clip
Voice	MiniMax Speech, F5 TTS	Varies

Why fal.ai?

600+ Models

Every major image, video, and audio model in one place

No GPU Setup

No servers to manage. Pay only for what you generate.

One API

Your Appie calls one endpoint for all media types

Fast

Optimized inference, results in seconds

RunPod for Heavy Workloads

For high-volume or custom workflows, RunPod lets you run your own Stable Diffusion, ComfyUI, or custom models on GPU instances from \$0.20/hour. Spin up, generate, spin down. No idle costs.

- Batch image generation (100s of images)
- Custom fine-tuned models
- ComfyUI workflows
- Video generation at scale

Setup

```
# Install fal.ai client
npm install @fal-ai/client

# Set your API key
openclaw config set services.fal.apiKey YOUR_KEY

# Your Appie can now generate images
# "Generate a banner for my new product launch"
```

Codex: Setup & Real Workflow

Use Codex as Appie's coding subagent for implementation work, while Appie (Opus) plans, reviews, and ships the result.

Setup in 3 Steps

```
# Step 1: Add Codex auth
openclaw auth add
# Choose "OpenAI Codex", follow OAuth
flow

# Step 2: Set as default coding model
# In openclaw.json:
{
  "agents": {
    "defaults": {
      "codingModel": "openai-codex/codex-5.5"
    }
  }
}

# Step 3: Done. Appie now uses
# Codex for all coding subagents.
```

Real Workflow Example

You: "Build me a landing page with a hero section, feature list, and CTA button. Use Tailwind CSS."

Appie (Opus): "Got it. I'll spawn Codex to write the code."

Appie spawns Codex subagent, passes requirements

Codex: Writes `pages/index.tsx`, hero section, feature cards, CTA button, all styled with Tailwind.

Appie (Opus): Reviews the code. "Looks clean. I'll save it to the repo and deploy to Vercel."

Appie: "Landing page deployed: `yoursite.vercel.app`"

You didn't write a single line of code.

Chapter 14

The Core Skill Set

A curated subset of the 155-skill Appie Kit. These are the skills that ship with every Appie by default.

Agent infrastructure (10)

brain-sync, project-context-sync, hermes-agent, mcp, git-sync, gitclaw, dogfood, autonomous-ai-agents, clawlist, gws-modelarmor

Design & creative (8)

ui-ux-pro-max, design, design-mastery, design-system, ui-styling, brand, creative, banner-design

Content production (8)

appie-video-production, content-safe-zones, slides, thumbnails, gifs, social-media, fal-ai, inference-sh

Productivity & ops (8)

productivity, note-taking, notion-masterclass, notion-tasks, n8n, smart-home, leisure, gaming

Development & devops (8)

software-development, coding, devops, infrastructure, mlops, github, read-github, digital-ocean

Research & data (5)

research, data-science, exa-plus, web-scraping-javascript-sites, feeds

Web & frontend (4)

webflow, nextjs-expert, playwright, diagramming

Email & communication (3)

email, google-workspace, google-drive

What's NEW between v4.4 and v4.5

Token reduction system

4-layer compression pattern for keeping context under control over long sessions. Per-message summarisation, tool-result truncation, memory flush before compaction, JIT skill-doc retrieval.

beads (graph task tracker)

Steve Yegge's distributed graph issue tracker for AI agents, backed by Dolt. Replaces messy markdown TODO lists with dependency-aware task graphs. `brew install beads`, then `bd init`.

agent-skills bundle

Production-grade engineering skills curated by Google's design lead (Addy Osmani). Selectively copy what fits your stack.

axon (graph code-intel)

Graph-based code intelligence that maps symbols, references, call graphs, and impact zones. `axon callers <symbol>`, `axon impact <file>:<line>`, `axon graph <package>`.

Fleet skill propagation via appie-brain

60-min cron that pulls latest from your private appie-brain repo. Skills only - never sync `.env.secrets` or per-Appie memory.

Hermes auto-skill-creator

Auto-creates a skill file when Hermes detects 5+ successful tool calls in sequence, error recovery, or user correction. Drops to `~/hermes/skills/auto/`. Review weekly.

Install commands

```
# OpenClaw
openclaw skills install <skill-name>
```

```
# Hermes Agent
hermes skills install <skill-name>
```

```
# Claude Code (just copy the file)
cp <skill>.md ~/.claude/skills/
```


Chapter 15

UI/UX Pro Max, Design Intelligence Skills

One of the most powerful skill sets in the Appie Brain. UI/UX Pro Max turns your Appie into a design expert with instant access to a massive library of design knowledge.

67+ UI Styles

Glassmorphism, brutalism, minimalism, neubrutalism, and more

161 Color Palettes

Organized by industry and mood

57 Font Pairings

With Google Fonts integration

99 UX Guidelines

Covering accessibility, animation, z-index, loading states

25 Chart Types

With library recommendations

How to Use It

```
# Search for design inspiration
python3 scripts/search.py "SaaS dashboard dark mode" --domain style

# Generate a complete design system
python3 scripts/search.py "fitness app modern bold" --design-system -p "FitApp"

# Find color palettes
python3 scripts/search.py "healthcare" --domain color

# Get font pairings
python3 scripts/search.py "elegant professional" --domain typography

# UX best practices
python3 scripts/search.py "animation accessibility" --domain ux
```

Real Example: Client Website Design

❏ **You:** "I need a design system for a luxury real estate brand. Modern, dark, elegant."

Appie searches the UI/UX Pro Max database → finds matching styles (dark luxury, editorial minimal) → selects color palette (#0A0A0A, #C9A96E, #F5F0EB) → picks fonts (Playfair Display + Inter) → generates a complete design system with component specs.

Result: Professional design system in 30 seconds. No Figma required for the initial direction.

Exa.ai, Google CLI & Ralph Wiggum

Three useful tools and workflows for making your Appie more capable: better search, Google Workspace automation, and autonomous coding loops.

Exa.ai: The Best Search Engine for AI Agents

Google search returns SEO spam. Exa.ai returns actual content. Neural search understands meaning, not just keywords. No SEO spam: results are real pages with real content.

Setup: Sign up at exa.ai, get an API key, add to your OpenClaw config with "provider": "exa". Done.

Example: "Find me Stripe's brand assets" → Exa finds Stripe's press kit directly. Google returns 10 SEO blog posts about "Top 10 Payment Processors." Exa wins.

Google CLI (gog): Manage Google Workspace from Terminal

If you use Gmail, Calendar, Drive, or Docs, you need `gog`. Install with `npm install -g gog`, authenticate with `gog auth login`.

Your Appie can check your Gmail inbox, add calendar events, upload files to Drive, and create/edit Google Docs, all via terminal commands.

Ralph Wiggum: Autonomous Coding Loops

A technique for running AI coding agents in autonomous loops. Define requirements in `PROMPT.md`, run Ralph, and the agent iterates until the task is done: writing code, running tests, fixing errors, retrying.

Install: `openclaw skills install ralph-loop` or `npm install -g ralph-starter`. Supports Claude Code, Codex, Gemini CLI, and OpenCode.

github.com/Th0rgal/open-ralph-wiggum (1,345 stars)

Chapter 18

Automation & Cron

Your Appie can work while you sleep. Heartbeats, cron jobs, and scheduled tasks turn your Appie from a chatbot into a true autonomous employee.



Heartbeats

A heartbeat is a regular check-in. Your Appie wakes up, checks things, then goes back to sleep. Default interval: 1 hour. Set model to Haiku for 90% cost savings.



HEARTBEAT.md

A short checklist (under 60 lines) that your Appie follows during each heartbeat. Check email, calendar, monitor services, write notes to daily log.



Cron Jobs

Schedule tasks at specific times. Morning brief at 8am. Security scan at 3am. Invoice check every Monday. Set it once, runs forever.



Git Sync

Auto-sync memory files to GitHub every 30 minutes. All Appies stay in sync automatically. Shared brain across your entire fleet.

Shared Brain: Git Sync Across Your Fleet

The shared brain is what makes a fleet of Appies feel like one coherent employee. Memory syncs automatically via GitHub: no API calls, no complex message passing. Just files in Git.

What to Sync

- MEMORY.md (long-term memory index)
- HEARTBEAT.md (shared checklist)
- VERSIONS.md (system state tracking)
- memory/YYYY-MM-DD.md (daily logs)
- memory/topics/*.md (organized knowledge)
- directives/*.md (SOPs, task assignments)
- SOUL.md (shared identity)

What NOT to Sync

- .env.secrets (API keys, tokens)
- openclaw.json (may contain secrets)
- *.token, *.key, *.pem (OAuth tokens, SSH keys)
- Personal data (anything sensitive)
- Session logs (too noisy, not useful)

The .gitignore

```
.env.secrets
.env
*.token
*.key
*.pem
node_modules/
.cache/
tmp/
session-logs/
```

The Weblyfe Fleet in Action

Here's how shared brain works in practice: three Appies collaborating seamlessly through shared markdown files.

01

Morning, Appie-1 (Mac)

You: "Check my email." Appie-1 checks Gmail, logs to `memory/2026-03-28.md`, commits, pushes to GitHub.

02

Afternoon, Appie-2 (VPS)

Appie-2 pulls repo, reads today's log. Sees urgent email from Client X. Searches social media for related mentions. Finds a tweet, adds to `memory/topics/client-x.md`, commits, pushes.

03

Evening, Appie-1 (Mac)

You: "What's new with Client X?" Appie-1 pulls repo, reads `memory/topics/client-x.md`. "Appie-2 found a tweet this afternoon. They're launching a new feature next week."

- ❏ You only talked to Appie-1. But Appie-2 contributed context. **Seamless collaboration.** Pro tip: Use branch names for experimental work. Main branch = stable memory. Feature branches = drafts, research.



Chapter 19

The Content Genome

Most content fails not because the idea is bad, but because the structure is wrong. The Content Genome breaks down every piece of content into its atomic parts so you can see exactly why something works, and replicate it on demand.

L1 Molecules

The atomic units of content: authority signals, tutorials, social proof, storytelling, and viral bait. Every post is built from at least one of these.

L2 Organisms

Combinations of molecules that form content types: case studies, founder stories, how-to guides, flex posts, and collabs. These are your repeatable formats.

L3 Ecosystem

Platform-content fit. What works on Instagram is not what works on LinkedIn. The ecosystem layer maps your organisms to the right distribution channel.

- 📄 Decompose any content piece into these atoms and you will immediately see what makes it work or fail. This is the framework behind every viral post in the Weblyfe library.



Chapter 20

Content Creation Ecosystem

Your Appie as Social Media Manager

Most people treat social media as a manual grind. Post, hope, repeat. Your Appie does it differently. This chapter covers the full content stack: from atomic content theory to a fully automated pipeline that posts, repurposes, and analyzes across 14 platforms while you sleep.

The Content Genome Framework

Zernio: One API for 14 Platforms

ManyChat + Notion Resource Delivery

The Automated Content Pipeline

Repurpose Engine: 1 Video = 12 Pieces

The Content Genome Framework

Every piece of content that goes viral is built from the same atomic elements. The Content Genome Framework breaks content into 4 levels, like DNA. Once you understand the atoms, you can reverse-engineer any post, reel, or article to find exactly what made it work or fail.

L0: Atoms

The raw building blocks of any content piece.

- Hook
- Face (on-camera presence)
- Motion (movement, cuts, transitions)
- Text Overlay
- Audio (music, voice, SFX)
- Emotion (what feeling does it trigger)
- Palette (color grading, brand colors)
- Pacing (fast cuts vs slow burn)

L1: Molecules

Atoms combined into recognizable content patterns.

- Authority (I know more than you)
- Tutorial (let me show you how)
- Social Proof (look what happened)
- Storytelling (here is what I went through)
- Viral Bait (share this or you will regret it)

L2: Organisms

Full content formats built from molecules.

- Case Study
- Founder Story
- How-To Guide
- Flex Post
- Collab / Duet

L3: Ecosystem

Platform-content fit. The same organism performs differently depending on where you post it.

- Instagram: emotion, face, motion, fast pacing
- LinkedIn: authority, storytelling, text-heavy
- YouTube: tutorial, long-form, high retention hooks
- Twitter/X: viral bait, threads, hot takes

❏ How to use this: Take any post that performed well. Strip it down to its atoms. Which hook type? Which emotion? Which molecule pattern? Now rebuild that formula with your own content. Your Appie can do this analysis automatically.

Zernio: One API for 14 Platforms

Zernio gives your Appie a single API to post, schedule, and analyze content across 14 social platforms. No more logging into each platform manually. One command, everywhere.



Setup

```
npm install -g @zernio/cli
zernio auth:login
# Connect your accounts via OAuth in the browser
# Each platform opens an auth flow, approve and done
```

Post to Multiple Platforms

```
zernio posts:create \
  --text "Your caption here" \
  --accounts ,, \
  --scheduledAt "2026-04-01T09:00:00Z"
```

Key Features

- Analytics: pull engagement, reach, and follower data per platform
- Media upload: attach images, videos, carousels per post
- Best posting times: Zernio suggests optimal schedule per platform
- Bulk scheduling: queue an entire week in one command

Free Tier

20 posts per month. Enough to test and validate your pipeline before committing.

Build Tier

\$19/month. 120 posts per month. This is the tier your Appie runs on.

ManyChat + Notion Resource Delivery

You do not need a landing page. You need a comment. ManyChat turns Instagram comments into automated DM flows that deliver your resources directly to anyone who asks. Your Appie can trigger these flows via API.

01	02	03
Set Your Trigger Keyword Pick a keyword like TechWiz, Guide, or Free. Tell your audience: comment this word to get the resource. ManyChat watches for it automatically.	ManyChat Auto-DMs the Link The moment someone comments your keyword, ManyChat fires a DM with your Notion link. No delay, no manual work, no missed leads.	Notion Delivers the Resource Your Notion page is shared publicly. It holds your video, guide, checklist, or template. No hosting required. No landing page builder. Just a Notion URL.

04
Your Appie Triggers Flows via API Your Appie can push content to ManyChat subscribers programmatically. New blog post? New product? One API call broadcasts it to your entire subscriber list.

- ❏ Why this works: Instagram's algorithm rewards comment engagement. Every person who comments your keyword boosts your post's reach. You get leads AND distribution at the same time.

Appie API Trigger Example

```
# Send a broadcast to all ManyChat subscribers
curl -X POST https://api.manychat.com/fb/sending/sendContent \
-H "Authorization: Bearer YOUR_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "subscriber_id": "{{subscriber_id}}",
  "data": {
    "version": "v2",
    "content": {
      "messages": [{
        "type": "text",
        "text": "Your new resource is live: https://notion.so/your-page"
      }]
    }
  }
}'
```

The Automated Content Pipeline

This is the full end-to-end flow. From a Notion card to a live post on 14 platforms, with zero manual steps. Your Appie orchestrates every stage.

01	02	03
Create in Notion Add your content to the Notion Content Factory. Fill in title, caption, media link (Google Drive), target platforms, and set status to Ready To Post.	n8n Detects the Change n8n watches your Notion database. The moment status changes to Ready To Post, it pulls the title, caption, and media URL automatically.	Claude Optimizes Per Platform Your Appie rewrites the caption for each platform using Content Genome rules. LinkedIn gets authority framing. Twitter gets a punchy hook. Instagram gets emotion and a CTA.
04	05	06
Media Gets Processed ffmpeg handles aspect ratio conversion, compression, and thumbnail generation. 16:9 for YouTube, 9:16 for Reels, 1:1 for Instagram feed. All automated.	Zernio Schedules Everything The processed media and optimized captions go to Zernio. Posts are scheduled at optimal times per platform. One API call, 14 platforms queued.	Notion Gets Updated Once posts go live, n8n writes the post URLs back to Notion and sets status to Done. Your content calendar stays accurate automatically.
07		
Weekly Analytics Pull Every Sunday, your Appie pulls engagement data from Zernio and writes a performance summary back into Notion. You wake up to a full report.		

The n8n Trigger (Notion Watcher)

```
// n8n Notion trigger node config
{
  "resource": "databasePage",
  "operation": "getAll",
  "databaseId": "YOUR_NOTION_DB_ID",
  "filterType": "manual",
  "filters": {
    "conditions": [{
      "key": "Status",
      "condition": "equals",
      "value": "Ready To Post"
    }]
  },
  "pollTimes": {
    "item": [{ "mode": "everyMinute" }]
  }
}
```


Repurpose Engine: 1 Video = 12 Pieces

You record once. Your Appie ships twelve. Every long-form video is a content goldmine. The Repurpose Engine extracts, reformats, and distributes every usable moment across every platform automatically.

What You Record

One long-form video. A tutorial, a podcast, a founder story, a product walkthrough. Anything over 10 minutes works.

What You Get

- **3 Reels/Shorts:** best 30-60s clips, reframed to 9:16 with face tracking
- **1 LinkedIn Article:** transcript cleaned up, insights added, published as thought leadership
- **3 Twitter Threads:** key takeaways broken into numbered thread format
- **2 Instagram Carousels:** step-by-step breakdowns, slide-by-slide
- **1 Quote Graphic:** the most impactful line, designed and branded
- **2 Stories:** behind-the-scenes moments, raw and authentic

The Tools

Remotion

Programmatic video editing in React. Your Appie generates video compositions from code. No Premiere, no After Effects. Just JavaScript.

```
npm install remotion
```

ffmpeg + Face Tracking

Extract clips, reframe to 9:16, track the speaker's face automatically. The clip extractor finds the best moments by analyzing audio energy and transcript keywords.

```
# Extract a clip and reframe to 9:16
ffmpeg -i input.mp4 \
-vf "crop=ih*9/16:ih:(iw-ih*9/16)/2:0" \
-t 60 output_reel.mp4
```

Voice DNA

Your Appie writes captions and articles in your authentic tone. Feed it 10 examples of your writing. It learns your voice and never sounds like a robot.

- ❑ The rule: never post raw long-form content to short-form platforms. Always repurpose. A 20-minute YouTube video has at least 12 pieces of content inside it. Your Appie finds them all.

Ready to build your content machine? Join the Weblyfe community at weblyfe.ai/community. Share your pipeline, get feedback, and connect with other builders running Appie-powered content systems.

Chapter 21

Troubleshooting

When things break, here's how to fix them. Start with `openclaw doctor`: it diagnoses and auto-fixes 90% of issues.

`openclaw doctor`

Checks Node.js version, config files, API keys, channel connections, installed skills, and gateway status. Shows exactly what's wrong.

```
openclaw doctor
```

`openclaw doctor --fix`

Auto-repairs missing directories, broken symlinks, permission issues, and outdated skills. Can't fix API keys (you have to set those), but handles most common problems.

```
openclaw doctor --fix
```

`openclaw status`

Full system health check: gateway status, model, uptime, channels, memory file sizes, heartbeat info, and cron jobs. Your dashboard at a glance.

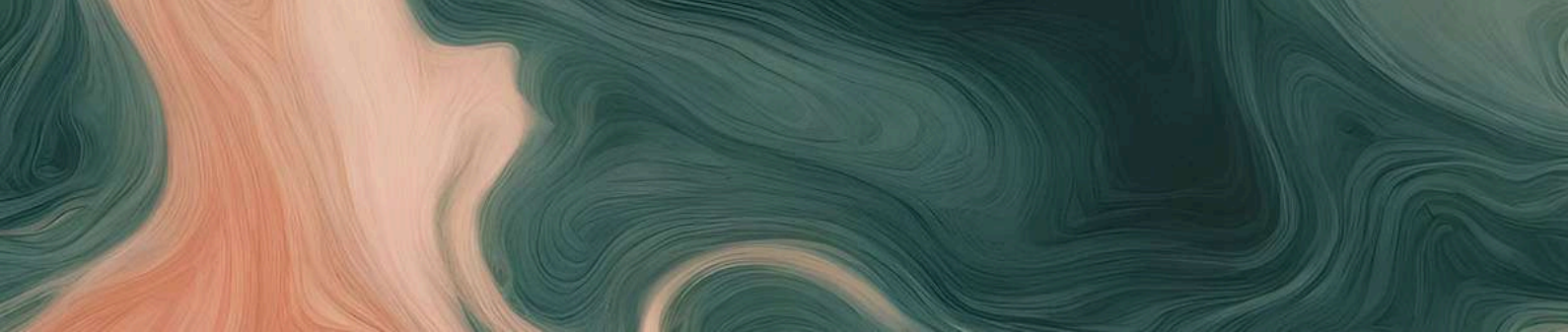
```
openclaw status
```

Common Issues & Fixes

If something isn't working, use this quick troubleshooting guide to identify the problem and apply the right fix.

Problem	Fix
Gateway won't start (port in use)	Run <code>lsof -i :3000</code> , kill the process, or use <code>openclaw gateway start -port 3001</code>
Bot not responding	Check token in <code>openclaw.json</code> , verify <code>allowFrom</code> has your numeric user ID, restart gateway
Token expired (401 error)	Run <code>openclaw auth refresh</code> and follow the OAuth flow again
Context too large	Archive old memory files, trim <code>MEMORY.md</code> to under 150 lines, send <code>/reset</code>
Rate limited	Switch to higher tier model, use OAuth subscription instead of API keys, slow down heartbeat interval
Slow responses	Trim <code>SOUL.md</code> + <code>MEMORY.md</code> , less context = faster responses
SSH kills gateway	Use <code>systemctl --user restart openclaw-gateway</code>

📌 **When all else fails:** Use the Google CLI (`gog`) directly to check email, read messages, and add calendar events, bypassing Appie entirely. Keep `gog` set up even if Appie handles it 99% of the time. It's your backup.



Chapter 22

Real Prompt Library

Here are 20 production-tested prompts you can copy-paste directly. These are organized by category: business operations, content & marketing, development, and personal productivity.

1

Business Operations

Morning briefs, client onboarding, invoice follow-ups, lead capture, competitor monitoring

2

Content & Marketing

Blog posts from topic to published, social media calendars, email nurture sequences, SEO audits

3

Development

Code reviews, Vercel deployments, database migrations, security audit scans

4

Personal Productivity

Weekly planning, meeting preparation, research synthesis, decision framework analysis

Business Operations Prompts

Use these production-tested prompts as a quick reference for common business operations workflows.

Morning Brief

Check my email, calendar, and weather for today.

Give me a concise morning brief:

- Urgent emails (if any)
- Meetings in the next 8 hours
- Weather (if rain, mention it)
- Top 3 priorities based on calendar and recent context

Keep it under 200 words.

Client Onboarding Automation

We have a **new** client: [Client Name].

Set up their onboarding:

1. **Create** a Google Drive folder
2. **Create** a project tracker doc
3. **Add** a kickoff **call** to calendar
4. Send a welcome email
5. **Add** them **to** CRM (Airtable)

Report back **when** done.

Invoice Follow-Up

Check our invoice **tracker** (Google Sheet: "Invoices 2026"). Find invoices that are:

- Overdue by **7+** days
- Not marked **as** "Paid"

For each, draft a polite follow-up email.

Send drafts to me **for** approval before sending.

Competitor Monitoring

Research our top 3 competitors: [A], [B], [C].

For each, find:

- Recent blog posts (last 30 days)
- New features or product updates
- Pricing changes
- Social media activity

Save to memory/topics/competitor-intel.md.

Highlight anything urgent.

Content, Development & Productivity Prompts

Use this as a clean document reference for production-tested prompts across content, development, and productivity workflows.

Blog Post: Topic to Published

Write a blog post: "[Topic]"

Steps:

1. Research (use Exa search)
2. Outline (intro, 3 sections, conclusion)
3. Draft (800-1200 words, conversational)
4. Add SEO metadata
5. Save as markdown
6. Publish to Webflow

Report back with the live URL.

Weekly Planning Session

It's Sunday evening.

Help me plan next week.

1. Review my calendar (Mon-Fri)
2. List all meetings and commitments
3. Identify "focus time" blocks
4. Suggest 3 priorities for the week
5. Draft a weekly plan

Ask clarifying questions if needed.

Code Review & Refactoring

Review the code in: [file or directory]

Look for:

- Bugs or logic errors
- Performance issues
- Code duplication (DRY violations)
- Missing error handling
- Security issues

Provide findings and suggest refactorings. Flag critical issues immediately.

Decision Framework Analysis

I'm deciding whether to: "[Decision]"

Help me think through it:

1. What are the options?
2. Pros and cons of each
3. What are the risks?
4. Cost (time, money, opportunity)?
5. What do I need to know to decide?
6. Recommendation

Save to: decisions/[decision-slug].md



Chapter 23

The Appie Kit

I built a starter kit for you. It's free, open-source, and ready to clone. Zero secrets: everything is safe to clone publicly. You add your own API keys after.

GitHub repo: github.com/S3YED/appie-kit

Templates

SOUL.md, USER.md, AGENTS.md - pre-written, ready to customize. Weblyfe-tested in production.

Skills

A curated set of skills: weather, email, calendar, web search - pre-installed and ready to go.

Tools

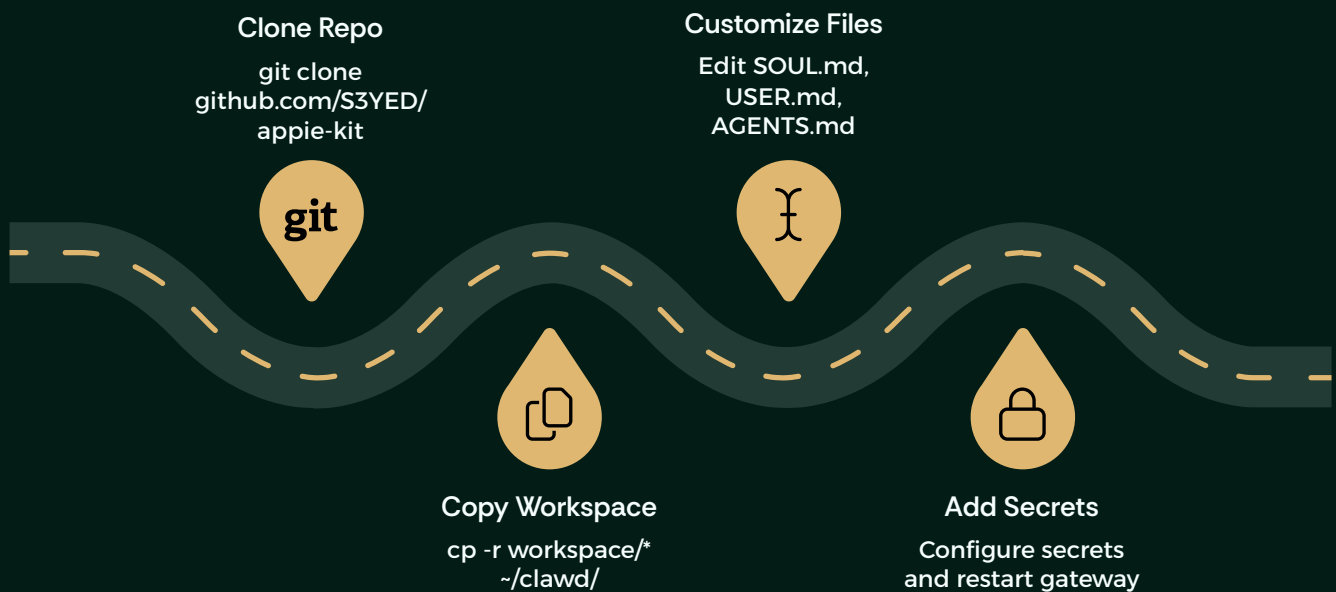
Scripts I actually use: Instagram poster, GitHub sync, Drive backup, Exa search wrapper.

Configs

Sample `openclaw.json`, `.gitignore`, and directory structure. Rename, add secrets, go.

How to Use the Appie Kit

Use the Appie Kit to get your Appie running quickly with a clean, repeatable setup. In a few simple steps, you'll clone the repo, copy the workspace files, customize your documents, and add your secrets before restarting the gateway.



In under 10 minutes, your Appie is running the Weblyfe-tested setup, the same configuration that powers three production Appies handling email, social media, code deployments, and client work.

📌 **Community:** Join the OpenClaw Discord at discord.com/invite/clawd. Share your setup, ask questions, contribute skills. The community is active and helpful. Pro tip: Fork the Appie Kit repo, customize it for your needs, then share it. Help the next person.

Chapter 24

Resources & Community

You made it. You now have a running AI employee. Here's where to go next: the channels, communities, and resources that will help you get the most out of your Appie.



Instagram: @seyed.jpg

Quick tips, memes, daily life, Appie updates. Behind-the-scenes of building with OpenClaw.



YouTube: @weblyfenl

Full tutorials, walkthroughs, deep dives. Watch me build stuff live. youtube.com/@weblyfenl



LinkedIn: Seyed Hosseini

Professional updates, case studies, thought pieces on AI + automation.

linkedin.com/in/seyedhosseini



Website: weblyfe.ai

Custom AI infrastructure, consulting, managed Appie setups. weblyfe.ai | hello@weblyfe.ai

Essential GitHub Repos & Recommended Reading

This reference table highlights the most useful GitHub repos and reading materials to help you explore, build, and optimize with Appie.

Repo	Stars	What
<u>openclaw/openclaw</u>	338k	The framework itself
<u>builderz-labs/mission-control</u>	3.5k	Fleet dashboard (manage multiple Appies)
<u>ThOrgal/open-ralph-wiggum</u>	1.3k	Autonomous coding loops
<u>swarmclawai/swarmclaw</u>	178	Multi-agent orchestration
<u>S3YED/appie-kit</u>	TBD	Drag-and-drop Appie setup (this guide's companion)

OpenClaw Architecture Visual Guide

[globalbuilders.club](#) : Beautiful visual breakdown of how OpenClaw works under the hood. Great for understanding the architecture.

"I Traced Every Token" (Cost Optimization)

Medium article by someone who traced every API call and reduced costs by 80%. Essential reading if you're running multiple Appies.

Ralph Wiggum Technique

[github.com/ThOrgal/open-ralph-wiggum](#) : Autonomous coding loops. Let your Appie iterate on code until it works.

Chapter 25

Move Your Orchestrator to Claude Code

If you have been running OpenClaw or Hermes on your Mac, you can now run your main 'brain' on Claude Code instead. We did this with Appie-1 in April 2026 and it changed what the orchestrator could do. This chapter is the walkthrough, written so you don't need to understand every command. You give Claude the prompts, Claude does the work.

Why Bother?

Sharper reasoning - Claude Opus 4.8 (Claude Code's default for complex tasks) plans better than OpenRouter-routed setups. The orchestrator's job is "think well, dispatch right", quality matters more than cost here.

More tools out of the box - Claude Code ships with 50+ tools (read files, edit files, run commands, search the web, spawn subagents, manage tasks). The orchestrator's job benefits from breadth.

Skills + slash commands feel native - Type `/morning-brief` in Claude Code and your morning-brief skill runs. Type `/research` and your research skill runs. Zero friction. 80+ skills wired into Claude Code on Appie-1 right now.

What You Keep, What You Change

Keep on OpenClaw or Hermes: Appie-2, Appie-3, Appie-4, and any worker Appies running on Hetzner or other VPS boxes. They run 24/7 doing scheduled work. OpenClaw and Hermes are the right shape for that.

Change to Claude Code: the orchestrator on your local machine, the brain you sit at and talk to interactively. That is where Claude Code wins.

i The Weblyfe v4.5 fleet pattern: Claude Code on the orchestrator, Hermes/OpenClaw on the workers.

Step 1 – Install Claude Code

```
curl -fsSL https://claude.ai/install.sh | bash
claude --version
# Should show version 0.32 or higher
```

Step 2 – Open a Persistent Terminal Session

```
brew install tmux
tmux new-session -d -s appie-1-brain
tmux attach -t appie-1-brain
# Press Ctrl-B then D to detach (session keeps running)
# Reattach: tmux attach -t appie-1-brain
```

Tip: Auto-start on reboot: ask your Claude (after Step 3) to set up a launchd plist so this session starts automatically every time your Mac boots.

Step 3 – Start Claude Code and Give It Your Context

Inside the tmux session, type: `claude`

You're going to be my orchestrator from now on, replacing my **OpenClaw** setup on this machine. I have an existing **OpenClaw** config at `~/.openclaw/` with a **SOUL.md**, memory files, and skills. I want you to:

1. Read `~/.openclaw/agents/main/SOUL.md` and copy its content to `~/.claude/CLAUDE.md`
2. Create `~/.claude/projects/-Users-appie-clawd/memory/` and copy everything from `~/.openclaw/memory/` into it.
3. List my **OpenClaw** skills and copy each **SKILL.md** to `~/.claude/skills/<skill-name>.md`
4. After you're done, list back: what's in **CLAUDE.md** (first 20 lines), how many memory files you copied, how many skills you ported, and tell me what you remember about me from **CLAUDE.md**.

Step 4 – Hook Up Telegram (ccgram or Claude Channels)

ccgram is a small program that turns your Telegram bot into Claude Code's mouth and ears.

Install the ccgram Telegram bridge for me:

1. Clone <https://github.com/S3YED/ccgram> into `~/clawd/tools/ccgram`
2. Run `npm install` inside that folder
3. Run `npm link` to expose the ccgram command globally
4. Run "`ccgram init`" interactively — stop and ask me for:
 - Telegram bot token (I'll get from @BotFather)
 - My Telegram user ID (I'll get from @userinfobot)
 - The target tmux session name: `appie-1-brain`
5. Start ccgram inside the tmux session pane 1.
6. Verify by sending me a Telegram test message: "ccgram online"

❗ **Alternative:** Claude Channels (first-party): launch Claude Code with: `claude --channels plugin:telegram@claude-plugins-official` - Set `TELEGRAM_STATE_DIR=$PWD/.telegram` first. Both approaches work. ccgram is simpler to set up; Claude Channels gives tighter first-party integration.

Step 5 – Add Scheduled Work via Flows

Flows are YAML files that tell Claude Code 'do X every Y.' They live at `~/.claude/flows/`.

Set up the flows system for me:

1. Create `~/.claude/flows/`
2. Install: `npm install -g @anthropic-ai/claude-flow-runner`
3. Create a "weekly-summary" flow at `~/.claude/flows/weekly-summary.yaml` that triggers every Sunday at 19:00 local time.
4. Start the flow-runner in pane 2 of the `appie-1-brain` tmux session.
5. Test with: `flow-runner test weekly-summary`

Step 6 – Verify the Full Setup

Do a full health check:

1. Confirm tmux session "`appie-1-brain`" is running with at least 3 panes.
2. Confirm ccgram is running in pane 1.
3. Confirm flow-runner is running in pane 2.
4. List the skills in `~/.claude/skills/` — count them.
5. Show me the first 30 lines of `~/.claude/CLAUDE.md`.
6. Send a Telegram message: "Setup complete. {N} skills loaded. {M} flows scheduled. Ready when you are."

Step 7 – Run the Old Setup in Parallel for Two Weeks

Don't kill OpenClaw yet. Let it keep running on the side. After two weeks of clean operation, stop OpenClaw on your Mac (the Hetzner workers stay on Hermes forever).

Stop the OpenClaw gateway on this Mac (don't uninstall yet):

1. Find the OpenClaw launchd plist at `~/Library/LaunchAgents/openclaw-gateway.plist`
2. Use `launchctl bootout` to stop the agent.
3. Confirm: `ps aux | grep openclaw` should show no running gateway.

The big shift in v4.5: you stopped writing bash commands and started giving prompts. The agent is the operator. You are the boss who delegates.

Free and Near-Free Token Sources

You don't need to spend \$80/mo on models to get started. Three legitimate paths to free tokens, in order of how we'd recommend stacking them.

Path 1 – OpenRouter Free Tier

OpenRouter offers 29 completely free models as of May 2026, no credit card required. Rate limits: roughly 20 requests/minute and 200 requests/day per model. Plenty for experimenting, prototyping, building your first Appie.

Model	Context	Notes
meta-llama/llama-3.3-70b-instruct:free	128K	GPT-4-level quality
nvidia/llama-3.1-nemotron-70b-instruct:free	128K	NVIDIA's tuned 70B, strong reasoning
deepseek/deepseek-r1:free	128K	Reasoning model, comparable to o1-mini
qwen/qwen-2.5-coder-7b:free	32K	Surprisingly good for code
google/gemma-3-27b-it:free	128K	Google's open model, solid daily driver
mistralai/mistral-7b-instruct:free	32K	Fast, good for routing layer

```
default_model: openrouter/meta-llama/llama-3.3-70b-instruct:free
fallback_chain:
  - openrouter/deepseek/deepseek-r1:free
  - openrouter/nvidia/llama-3.1-nemotron-70b-instruct:free
  - openrouter/google/gemma-3-27b-it:free
```

Path 2 – DeepSeek Native API (Paid but Very Cheap)

Once you're past free tier, DeepSeek's native API is the cheapest paid option. DeepSeek V4 Flash (May 2026): 1.05M context window at \$0.126 input / \$0.252 output per million tokens.

Model	Input (\$/M)	Output (\$/M)	Context	Notes
DeepSeek V4 Flash	\$0.126	\$0.252	1.05M	Daily driver, huge context
Kimi K2 (Moonshot)	\$0.074	\$0.350	262K	Better reasoning than V4 Flash
HY3 (Hunyuan)	\$0.066	\$0.26	262K	Cheapest paid option per token
DeepSeek R1	\$0.55	\$2.19	128K	Reasoning specialist

Note: The cache-discount on DeepSeek is the killer feature. If you keep your system prompt stable, repeat calls cost roughly 25% of the first call.

```
providers:
  deepseek:
    api_key: ${DEEPSEEK_API_KEY}
    base_url: https://api.deepseek.com/v1
    default_model: deepseek/deepseek-v4-flash
```

Path 3 – OpenAI Codex via ChatGPT Plus OAuth

If you already have a ChatGPT Plus subscription (\$20/mo), you have free access to Codex 5.5 for coding work. No additional API charges. Setup is one command:

OpenClaw:

```
openclaw auth add
# Choose "OpenAI Codex"
from the menu
# Browser opens, sign in to
ChatGPT, grant access
```

Hermes Agent:

```
hermes config set
agents.coding_model
openai-codex/codex-5.5
hermes auth add
# Same OAuth flow
```

Claude Code:

```
claude /auth add openai-
codex
# OAuth flow in browser
```

Recommended Starter Combo

01	02	03
ChatGPT Plus (\$20/mo) free Codex 5.5 via OAuth	OpenRouter free tier free reasoning + general work via Nemotron 70B + DeepSeek R1	Local Ollama for offline fallback (optional)

i Total: \$20/mo for a fully functional Appie. No other LLM bill. When you're ready to scale up, switch the default model to DeepSeek V4 Flash (\$5-15/mo for a single Appie at normal usage rates).

The Design + Dev Stack

The skills library covers what your Appie does. This chapter covers what your Appie uses when she builds frontend, animations, or designs for you.

21st.dev (the modern shadcn extension)

The v4.5 default for component installation. Hosts MagicUI, Aceternity, tweakcn, and 50+ other registries in a single CLI.

```
npx shadcn@latest add magicui:border-beam
npx shadcn@latest add aceternity:spotlight
npx shadcn@latest add tweakcn:pricing-cards-cta-1
hermes skills install 21st-dev
```

When the Appie uses it: customer asks for animated gradient text and floating particles, Appie picks from 21st.dev, composes the page in seconds, ships.

UI/UX Pro Max (design intelligence backbone)

67+ UI styles, 161 color palettes, 57 font pairings, 99 UX guidelines, 25 chart types. v4.5 adds: style memory (remembers which styles worked for past clients) and live Figma sync (lifts colour tokens + typography directly via the Figma API).

Design-Mastery (the v4.5 deep design skill)

Where UI/UX Pro Max is catalog (here are 67 styles, pick one), design-mastery is judgment (here is why a luxury real-estate brand should NOT use neubrutalism even though it's trendy). Contains: 30+ design principles from Apple HIG, Material 3, IBM Carbon. Anti-patterns. Brief-to-decision walkthroughs.

```
hermes skills install design-mastery
```

Framer Motion (the animation skill)

New v4.5 addition. Teaches scroll-triggered reveals, hero animations with useScroll + useTransform, spring-based interaction patterns (button bounce, card tilt, magnetic hover), coordinated multi-element timeline animations.

```
hermes skills install framer-motion
```

Real example: 'Add a hero section with a floating Appie character that tilts toward the cursor.' The skill knows to use useMotionValue for cursor X/Y, useSpring for smooth lerp, and the right damping ratios.

Remotion + GSAP (video + premium animation)

For programmatic video generation (announcement clips, repurposed YouTube to Reels): remotion-best-practices. For premium timeline animation (Apple keynote-style): gsap-mastery.

```
hermes skills install remotion-best-practices
hermes skills install gsap-mastery
```

The trio (Framer Motion / GSAP / Remotion) covers 95% of motion-design needs.

Recommended install order

When setting up a new Appie for design/dev work, install in this order:

```
hermes skills install 21st-dev
hermes skills install ui-ux-pro-max
hermes skills install design-mastery
hermes skills install framer-motion
hermes skills install nextjs-expert
hermes skills install gsap-mastery
hermes skills install remotion-best-practices
```

Six commands. Your Appie goes from 'code monkey' to 'design partner who codes.'

Chapter 28

The Voice Stack, TTS + STT for 2026

Your Appie should be able to speak and listen. v4.4 had this scattered across hints; v4.5 locks the voice stack with concrete model recommendations.

Speech-to-Text (STT), What Your Appie Uses to Hear You

faster-whisper (large-v3), the default:

What we use locally for transcribing voice messages from Telegram. Runs on CPU, accurate across most languages, handles accents and background noise well. Latency: roughly 5-10 seconds for a 30-second voice clip on a Mac Mini M4. Quality: gold standard.

```
pip install faster-whisper
```

```
from faster_whisper import WhisperModel
model = WhisperModel("large-v3",
device="cpu", compute_type="int8")
segments, info =
model.transcribe("audio.ogg",
language=None, vad_filter=True)
```

Distil-Whisper, when you need it faster:

Distilled version of Whisper, 6x faster, 49% smaller, 1% WER difference. Use it when latency matters more than 1% accuracy.

NVIDIA Parakeet, for English-only sub-second:

Released late 2025, optimised for sub-second STT on English. If your Appie is English-only and you want near-realtime, this beats Whisper.

Voxtral (Mistral), multilingual + open weights:

Mistral's late-2025 STT release. Multilingual, open weights (Apache 2.0), competitive with Whisper on European languages.

Text-to-Speech (TTS), What Your Appie Uses to Talk

Model	Speed	Best for
Kokoro (82M, Apache 2.0)	Sub-0.3s	Daily Telegram voice replies - v4.5 default
F5-TTS	~7s/para	Zero-shot voice cloning from 10-second sample
Fish Audio S2 Pro	1-2s/sent	Premium quality, branded content
Sesame CSM (1B)	Moderate	Two-speaker dialog, conversational Appies
Piper	Very fast	Embedded / offline / Raspberry Pi setups
Coqui XTTS-v2	Moderate	Multilingual cloning (v4.5 reference)
MiniMax Speech / ElevenLabs	Fast (API)	Paid premium, SLA-backed

Recommended Voice Stack for v4.5

STT: faster-whisper large-v3 (local, free, accurate, 5-10s latency)

TTS: Kokoro for daily replies (sub-0.3s, free, runs on CPU)

Voice cloning (optional): F5-TTS when the customer wants Appie to sound like a specific voice

```
pip install faster-whisper kokoro f5-tts
hermes skills install voice-stack
```



Note: The voice-stack skill ties STT-in / TTS-out together with your Telegram bridge so voice messages flow both ways automatically.

ElevenLabs Recipe (Premium Quality)

For cases where voice quality really matters, demos, branded content, client Appies, ElevenLabs remains the industry leader.

```
# ElevenLabs settings for Appie voice
voice_id = "N2IVS1w4EtoT3dr4eOWO" # Callum
model_id = "eleven_v3"
stability = 0.3
similarity_boost = 0.75
style = 0.0
use_speaker_boost = True
```

Chapter 29

The Appie Kit – The Open Source Companion

The exact configuration, tools, and skills extracted from running Weblyfe, a web design agency powered by AI employees. Free, MIT-licensed, production-tested on the real fleet.

github.com/S3YED/appie-kit

What's Inside

155 production skills across 8 categories. The same system that runs a real web design agency, now public.

Category	Skills	What it covers
automation	19	Agent self-management, fleet coordination, brain sync, sub-agent spawning, ClawFlow workflows
communication	6	Telegram, Discord, Slack, iMessage (BlueBubbles), WhatsApp (wacli), voice calls
content	49	UI/UX design, image gen, video production, SEO, copywriting, Next.js, TTS, stable diffusion
integrations	31	Airtable, Notion, Google Workspace, GitHub, n8n, fal.ai, Spotify, Webflow, Trello, Gemini
knowledge	22	LLM fine-tuning (LoRA/GRPO), quantization, vLLM, Whisper STT, DSPy, model evaluation
meta	8	Skill authoring, ClawHub registry, task planning, dogfood QA, brainstorming frameworks
ops	16	GitHub/git, DigitalOcean, tmux, 1Password, healthcheck, session management, fleet maintenance
personal	4	Bear Notes, Spotify, Things 3 (macOS), weather

6 Identity Files (Workspace Templates)

SOUL.md

Personality, values, communication style

USER.md

Who you are: timezone, preferences, context

TOOLS.md

Available tools and API references

AGENTS.md

Operating rules, safety, group chat behavior

IDENTITY.md

Multi-agent identity (for fleets)

HEARTBEAT.md

Proactive check-in configuration

Install

```
git clone https://github.com/S3YED/appie-kit.git
cd appie-kit
./install.sh /path/to/your/workspace/
```

Skills into Hermes:

```
cp -r skills/content/design ~/.hermes/skills/
cp -r skills/automation/* ~/.hermes/skills/
# or copy entire categories
```

Skills into OpenClaw:

```
cp -r skills/content/design ~/.openclaw/skills/
```

Also Included

Production shell tools

setup-openclaw-mac.sh, setup-openclaw-vps.sh, hermes-agent-install.sh, security-scan.sh, health-check.sh, session-manager.sh, safe-gateway-restart.sh

Platform configs

Telegram, Discord, WhatsApp, multi-agent example configs

Case studies

Lead capture automation, multi-agent fleet (3 Appies), content pipeline, desktop automation (Clawd Cursor)

Prompt library

Production-tested prompts across business, content, support, devops

The E-commerce Control Suite (In Development)

A future Appie Kit extension, three pillars for DTC operators and agencies: (1) Store operations: Shopify inventory, orders, returns, customer service. (2) Store creation: spin up a new Shopify store including theme, imagery, copy, shipping config, in under 60 minutes. (3) Paid acquisition: Meta/Google ad audits, creative fatigue detection, ROI tuning. Ships as an Appie Kit add-on on GitHub when ready.

Mem0 – Cross-Session Memory

Hermes has built-in session memory, but it doesn't remember what a user said last week. If someone told your Appie two weeks ago 'I prefer short replies in Dutch', a fresh Hermes session forgets that. Mem0 closes the gap.

What Mem0 Is

Mem0 ('mem-zero') is an open-source memory infrastructure layer that sits between your agent and its storage. When a conversation happens, Mem0 automatically extracts structured facts from the text using an LLM, stores them in hybrid storage (vector DB for semantic search, graph store for entity relations), and retrieves the most relevant facts at the start of each new session.

~90% fewer tokens
than replaying full
conversation history

26% higher quality score
than OpenAI's own memory
system (LLM-as-a-judge)

Separate namespace
Every user_id is a separate
namespace - memories
never leak between users

When to Use Mem0 vs Hermes Built-In Memory

Use Hermes built-in memory when:

- The task fits inside a single session or short conversation window
- You want zero extra infrastructure
- The agent handles one-shot tasks (code fixes, file edits, cron reports)

Use Mem0 when:

- You want your Appie to remember stated preferences, project context, or client details across days and weeks
- You're building a customer-facing bot where personalization matters
- You want semantic search across prior interactions
- Multiple agent instances should share the same memory store

Note: They are not mutually exclusive. Keep Hermes built-in memory for session continuity, layer Mem0 on top for cross-session long-term facts.

Setup – Option A: Cloud (Zero-Ops, Free Tier)

```
pip install mem0ai

from mem0 import MemoryClient

client = MemoryClient(api_key="m0-your_api_key") # via app.mem0.ai

# After a conversation exchange
client.add(
    messages=[
        {"role": "user", "content": "I prefer Dutch, always keep replies short."},
        {"role": "assistant", "content": "Understood."}
    ],
    user_id="seyed"
)

# Retrieve relevant memories before generating a reply
results = client.search(query=user_message, user_id="seyed", limit=5)
context = "\n".join(f"- {r['memory']}" for r in results["results"])
```

Setup – Option B: Self-Hosted OSS (Full Data Control)

```
pip install mem0ai
docker compose up -d # starts Qdrant + the Mem0 service

from mem0 import Memory

config = {
    "llm": {
        "provider": "litellm",
        "config": {
            "model": "deepseek/deepseek-v4-flash",
            "temperature": 0.1,
        }
    },
    "vector_store": {
        "provider": "qdrant",
        "config": {"host": "localhost", "port": 6333}
    }
}

memory = Memory.from_config(config)
memory.add(messages, user_id="seyed")
results = memory.search(query=user_message, user_id="seyed", limit=5)
```

Pricing

Tier	Price	Memories	Retrieval calls/mo
Hobby	Free	10,000	1,000
Starter	\$19/mo	50,000	5,000
Pro	\$249/mo	Unlimited	Unlimited + graph memory
Self-hosted OSS	Free	Unlimited	Unlimited

Migrate from OpenClaw to Hermes

We went through this with the full Weblyfe fleet in May 2026. This is the playbook based on what we learned.

When to Switch

Move to Hermes if you:

- Want to run multiple Appies on one machine (Hermes supports multi-instance via HERMES_HOME)
- Want cleaner multi-agent setup (Hermes has built-in delegation and subagent spawning)
- Want OpenRouter as your primary model provider (Hermes has native OpenRouter integration)
- Want the latest Hermes features (auto skill creation, session compression, Mem0 integration)

Stay on OpenClaw if:

- You have a stable working system and don't need to switch. "If it works, don't fix it" applies here too.

Step 1 - Install Hermes

```
pip install hermes-agent
# or for isolation:
pipx install hermes-agent

hermes --version
```

Step 2 - Move Your SOUL.md

```
cp ~/.openclaw/workspace/SOUL.md ~/.hermes/SOUL.md
```

Hermes auto-loads SOUL.md at the start of every session.

Step 3 - Move Skills

OpenClaw skills live at ~/.openclaw/skills/<name>/SKILL.md. Hermes skills live at ~/.hermes/skills/<name>/SKILL.md.

```
for skill_dir in ~/.openclaw/skills/*; do
  skill_name=$(basename "$skill_dir")
  mkdir -p ~/.hermes/skills/"$skill_name"
  cp "$skill_dir/SKILL.md" ~/.hermes/skills/"$skill_name"/SKILL.md 2>/dev/null || true
done
echo "Skills moved: $(ls ~/.hermes/skills/ | wc -l)"
```

Step 4 - Re-connect Telegram

Hermes uses the same TELEGRAM_BOT_TOKEN as OpenClaw.

```
# Add to ~/.hermes/.env
echo "TELEGRAM_BOT_TOKEN=your_bot_token" >> ~/.hermes/.env
echo "TELEGRAM_ALLOWED_USERS=your_user_id" >> ~/.hermes/.env
echo "OPENROUTER_API_KEY=your_openrouter_key" >> ~/.hermes/.env

hermes gateway run
```

Step 5 - Configure the Model

```
# ~/.hermes/config.yaml
model:
  default: deepseek/deepseek-v4-flash
  provider: openrouter
  base_url: https://openrouter.ai/api/v1
```

Step 6 - Start the Gateway and Test

```
hermes gateway run --replace
```

Send a test message from your Telegram bot to yourself. If a reply comes back, it works.

Step 7 - Stop OpenClaw (After Two Weeks of Stable Use)

```
launchctl unload ~/Library/LaunchAgents/openclaw-gateway.plist 2>/dev/null
```

Don't uninstall OpenClaw for at least 30 days of trouble-free Hermes operation. Or keep it as a fallback. No rush.

Multiple Hermes Instances (A Fleet on One Machine)

Each Appie instance gets its own HERMES_HOME directory:

```
# Appie-3 (CTO)
HERMES_HOME=~/.hermes-appie3 TELEGRAM_BOT_TOKEN=<token3> hermes gateway run

# Appie-4 (CFO)
HERMES_HOME=~/.hermes-appie4 TELEGRAM_BOT_TOKEN=<token4> hermes gateway run
```

Use launchd plists (macOS) or systemd units (Linux) to auto-start each one. The Appie Kit ships example plist templates.

Hermes Setup & Auth - Wire Up Telegram, OpenRouter, Codex

When you stand up a fresh Hermes Apple, four questions decide the day: (1) What model is the brain? (2) How does it authenticate? (3) Where does it listen? (4) What tools can it use?

The hermes setup Wizard

Run the full wizard the first time:

```
hermes setup

# Re-run a section after-the-fact:
hermes setup model      # change brain model / provider
hermes setup gateway    # change which channels are wired
hermes setup tools      # toggle built-in toolsets
hermes setup tts         # voice output settings
hermes setup agent      # max turns, reasoning effort, compression
hermes setup terminal    # CLI display preferences

hermes setup --quick     # only prompt for fields that are missing
hermes setup --non-interactive # use env vars and defaults (fleet provisioning)
hermes setup --reset     # start over
```

The hermes auth Command Tree

```
hermes auth
├── add    <provider> # add a credential (OAuth flow or API key prompt)
├── list   # list every stored credential, grouped by provider
├── status <provider> # show whether <provider> is logged in / exhausted
├── remove <id>      # remove a credential by index, id, or label
├── reset  <provider> # clear "exhausted" flags after a quota refresh
├── logout <provider> # log out completely and forget the auth state
└── spotify # PKCE-OAuth flow for the Spotify skill
```

Provider Matrix

Provider	Auth type	Recommended use	Add command
openrouter	API key	Budget tier, default driver	hermes auth add openrouter
openai-codex	OAuth (ChatGPT Plus)	Code-writing muscle, free with subscription	hermes auth add openai-codex --type oauth
anthropic	OAuth (Claude Pro/Max) or API key	Premium escalations, voice work	hermes auth add anthropic --type oauth
gemini	API key	Cheap long-context fallback	hermes auth add gemini
minimax	API key	Voice (MiniMax Speech), models	hermes auth add minimax
copilot	GitHub token	Code reviews via Copilot	hermes auth add copilot

Telegram, Wiring the Bot

Telegram needs three things: a bot token, a list of allowed user IDs, and optionally a pairing code flow.

1. Open @BotFather in Telegram. Send /newbot, follow the prompts. Save the token.
2. Open @userinfobot in Telegram, send any message, it replies with your numeric user ID.

```
# Add to ~/.hermes/.env
echo "TELEGRAM_BOT_TOKEN=your_token_here" >> ~/.hermes/.env
echo "TELEGRAM_ALLOWED_USERS=your_user_id" >> ~/.hermes/.env

# Or use the setup wizard:
hermes setup gateway
# Pick Telegram, paste token, paste user ID, done.

hermes gateway run --replace
```

Adding more users via pairing codes:

```
hermes pairing list      # see pending requests
hermes pairing approve <code> # approve a request
hermes pairing revoke <user_id> # revoke access later
hermes pairing clear-pending # nuke all pending pairings
```

OpenRouter, The Budget Driver

```
hermes auth add openrouter
# Get your key at https://openrouter.ai/keys
```

```
# ~/.hermes/config.yaml
model:
  default: deepseek/deepseek-v4-flash
  provider: openrouter
  base_url: https://openrouter.ai/api/v1
```

```
hermes auth status openrouter
# openrouter: logged in (env:OPENROUTER_API_KEY)
```

Codex via ChatGPT Plus, Free Code-Writing Muscle

```
hermes auth add openai-codex --type oauth
# Browser opens, sign in to ChatGPT, grant access
```

```
hermes auth status openai-codex
# openai-codex: logged in (oauth, expires in 2026-08-13)
```

```
# In ~/.hermes/config.yaml
delegation:
  muscle_model: openai-codex
```

Troubleshooting

Symptom	Fix
No Codex credentials stored	hermes auth add openai-codex --type oauth
OpenRouter quota exhausted	openrouter.ai/credits, then: hermes auth reset openrouter
Telegram bot doesn't reply	Check: ps aux grep hermes_cli grep gateway - then tail -50 ~/.hermes/logs/gateway.log
Gateway already running	hermes gateway run --replace

Daily-Driver Cheatsheet

```
hermes setup          # full wizard (first time)
hermes setup --quick  # only ask for what's missing
hermes auth add <provider> # add a credential
hermes auth list      # see everything stored
hermes auth status <provider> # is it logged in?
hermes auth reset <provider> # clear exhausted flag
hermes gateway run --replace # (re)start the gateway
```

Appendix

Hermes config.yaml – Production Reference

The minimal delta from the default `Hermes config.yaml`, only the settings worth overriding for a production Appie deployment.

```
agent:
  max_turns: 60
  gateway_timeout: 1800      # bump to 3600 only for long research jobs
  reasoning_effort: medium   # use /reasoning high on demand for complex tasks
  tool_use_enforcement: auto # correct for DeepSeek, on by default for GPT/Gemini

approvals:
  mode: smart                # reduces prompt fatigue, still blocks destructive cmds
  cron_mode: deny            # cron jobs must hard-fail, not auto-approve
  timeout: 60

compression:
  enabled: true
  threshold: 0.7             # was 0.5; let context fill more before compressing
  target_ratio: 0.2
  protect_last_n: 20

memory:
  memory_enabled: true
  user_profile_enabled: true

sessions:
  auto_prune: true           # keeps disk from growing unbounded
  retention_days: 30

security:
  redact_secrets: true       # hide API keys in logs

display:
  personality: helpful       # override "kawaii" for production/customer channels

delegation:
  max_spawn_depth: 1
  subagent_auto_approve: false # never true in production
```


The Five Settings That Matter Most



`model.default`

Largest lever. Wrong model = wrong cost on every interaction.



`reasoning_effort`

medium as the floor, use high on demand. Always-on high burns tokens.



`approvals.cron_mode: deny`

Never allow. Cron jobs must hard-fail when uncertain.



`security.redact_secrets: true`

Off by default. Always on in production.



`compression.threshold: 0.7`

Up from 0.5. Fewer helper-model calls.

Build it. Tweak it. Make it yours.

v4.5 is the last update at the €65 price point. The next big revision (v5) will be a separate product with deeper architectural changes. Lifetime updates on your current purchase still cover every minor revision after this one (v4.6, v4.7, and on).

[Instagram · @seyed.jpg](#)

[YouTube · @weblyfenl](#)

[LinkedIn · Seyed Hosseini](#)

[Website · weblyfe.ai](#)

[The Kit · github.com/S3YED/appie-kit](#)

A Personal Note

Before we close, I want to share something personal.

I'm working on a project close to my heart: building permanent homes for 10 orphaned Afghan families. These families lost everything. I want to give them a place to rebuild.

If this guide helped you, and if you're in a position to give, consider supporting this project:

[Support on GoFundMe](#)

[Learn About Weblyfe](#)

Every contribution helps. If you can't give, sharing the link helps too. Building Appie is about abundance. Building these homes is about abundance too. Let's expand abundance everywhere we can.



Weblyfe Services

If you want help setting up your Appie, Weblyfe offers three tiers of support: from fully managed setups to strategic consulting.

Managed Setup

I'll set up your entire Appie infrastructure (single or fleet), customized for your workflows. You just message your bot when it's ready.

Custom AI Infrastructure

Multi-Appie fleets, integrations with your tools (CRM, project management, custom APIs), full automation pipelines.

Consulting

Strategy sessions, architecture design, workflow optimization. I'll help you figure out what to build and how.

Contact: weblyfe.ai | hello@weblyfe.ai

LIVE SESSION – COMING SOON

The next live session date will be announced to PDF buyers via email. Stay tuned.

What Seyed will show:

- Set up Hermes Agent from scratch
- Load all skills and configure your agent
- Build funnels, brands, landing pages with AI
- Automate leads, support, and messaging
- Live Q&A

📅  Date: To be announced | 📍 Location: TBD | 👥 Who: PDF buyers only | 💻 Bring: Your laptop

Details and RSVP sent via email.

Final Thoughts

You're not using AI. You're building with it.

Appie isn't a tool you use once and forget. It's a system you refine, a partner you collaborate with, an employee you train. The more you use it, the better it gets. The more you teach it, the more capable it becomes. The more you trust it, the more it frees you to focus on what matters.

This is the future of work. Not AI replacing humans. **AI working for humans.**

Build your Appie. Make it yours. Watch it transform how you work. And once it's working, **teach someone else.**

The future is abundant. **Let's build it together.** Welcome to the future.

Build Your Own Appie v4.5, By Seyed Hosseini | [Weblyfe.ai](https://weblyfe.ai) | May 2026 | Digital Excellence

The Appie Cheatsheet

Print this. Pin it. Thank us later.

Setup & Lifecycle

Command	What It Does
<code>npm i -g openclaw</code>	Install OpenClaw globally
<code>openclaw onboard</code>	Interactive setup wizard (fixes 99% of issues)
<code>openclaw doctor</code>	Diagnose all problems
<code>openclaw doctor --fix</code>	Auto-fix what it finds
<code>openclaw gateway start</code>	Start your Appie
<code>openclaw gateway stop</code>	Stop your Appie
<code>openclaw gateway restart</code>	Restart (use after config changes)
<code>openclaw status</code>	System overview + session info
<code>openclaw status --deep</code>	Deep probe (channels, auth, everything)
<code>openclaw update</code>	Update to latest version
<code>openclaw backup --compress</code>	Snapshot your agent state
Hermes Agent Equivalents	
<code>hermes setup</code>	Interactive setup wizard
<code>hermes doctor</code>	Diagnose all problems
<code>hermes doctor --fix</code>	Auto-fix what it finds
<code>hermes gateway run</code>	Start your Appie
<code>hermes gateway stop</code>	Stop your Appie
<code>hermes gateway run --replace</code>	Restart (replaces running instance)
<code>hermes status</code>	System overview + session info
<code>hermes auth</code>	Manage authentication

In-Chat Commands & Model Switching

In-Chat Commands (send to your Appie)

Command	What It Does
<code>/status</code>	Token usage, model, context %
<code>/model sonnet</code>	Switch to Sonnet (cheaper)
<code>/model opus</code>	Switch to Opus (smarter)
<code>/model codex</code>	Switch to Codex (free coding)
<code>/model haiku</code>	Switch to Haiku (cheapest)
<code>/reset</code>	Clear session history (keeps memory files)
<code>/reasoning on</code>	Enable extended thinking
<code>/reasoning off</code>	Disable extended thinking

Model Cheatsheet

*Free via ChatGPT Plus subscription

Pro move: Set Haiku for heartbeats, Codex for coding, Opus for you.

Alias	Best For	Cost
<code>opus</code>	Strategy, architecture	\$\$\$
<code>sonnet</code>	Daily work, coding	\$\$
<code>haiku</code>	Heartbeats, monitoring	\$
<code>codex</code>	Heavy coding	Free*
<code>gpt</code>	General tasks	\$\$
<code>gemini</code>	Research, long context	\$\$

Find & Install Anything

Skill Commands

What You Want	Command
Browse all skills	<code>openclaw skills list</code>
Install a skill	<code>openclaw skills install <name></code>
Install from GitHub	<code>openclaw install user/repo</code>
Search ClewHub	Visit clawhub.com
Install Ralph loops	<code>openclaw skills install ralph-loop</code>
Install web search	<code>openclaw skills install web-search</code>
Install browser use	<code>openclaw skills install browser-use</code>
Install coding agent	<code>openclaw skills install coding-agent</code>

Power Installs

```
# Mission Control dashboard
git clone https://github.com/builderz-labs/mission-control
cd mission-control && pnpm install && pnpm build

# Ralph Wiggum (autonomous coding)
npm install -g ralph-starter

# SwarmClaw (multi-agent)
git clone https://github.com/swarmclawai/swarmclaw
cd swarmclaw && pnpm install

# Appie Kit (drag-and-drop setup)
git clone https://github.com/S3YED/appie-kit
# Copy workspace/ into your OpenClaw dir

# Fleet CLI (multi-gateway management)
openclaw install oguzhnatly/fleet
```



Config One-Liners & Security

Config One-Liners

```
# Set default model
openclaw config set \
  agents.defaults.model.primary \
  "anthropic/claude-sonnet-4-6"

# Set heartbeat interval
openclaw config set \
  agents.defaults.heartbeat.every "1h"

# Set heartbeat to cheap model
openclaw config set \
  agents.defaults.heartbeat.model \
  "anthropic/claude-haiku-4-6"

# Add Telegram bot
openclaw config set \
  channels.telegram.botToken "YOUR_TOKEN"

# Lock bot to your ID only
openclaw config set \
  channels.telegram.allowFrom \
  ["YOUR_TELEGRAM_ID"]

# Enable Exa search
openclaw config set \
  tools.web.search.provider "exa"
```

```
# Hermes Agent equivalents

# Set default model
hermes config set model.primary
"anthropic/claude-sonnet-4-6"

# Set heartbeat interval
hermes config set heartbeat.every "1h"

# Add Telegram bot
hermes config set channels.telegram.botToken
"YOUR_TOKEN"

# Lock bot to your ID only
hermes config set channels.telegram.allowFrom
["YOUR_TELEGRAM_ID"]
```

Security Essentials

```
# Lock your secrets file
chmod 600 .env.secrets

# Run security scan
openclaw security audit

# Deep security probe
openclaw security audit --deep
```

The 3 Rules

1. Gateway on localhost or Tailscale only, NEVER public
2. allowFrom on every channel, lock to your ID
3. API spending limits set at provider dashboard

Cron & Automation (In-Chat)

"Remind me in 20 minutes to check the deploy"

"Every morning at 8am, check my email and calendar and send me a brief"

"Run a security scan every day at 3am and alert me if anything is wrong"

SOS Troubleshooting Speed Run

Quick fixes for the most common issues, OpenClaw and Hermes Agent.

Problem	Fix
Gateway won't start	<code>ls -l :19337</code> then <code>kill <PID></code>
Bot not responding	Check <code>allowFrom</code> has your numeric ID
Token expired	<code>openclaw auth add</code> (re-authenticate)
Context too large	Send <code>/reset</code> to clear session
Rate limited	<code>/model haiku</code> to switch to cheaper model
Slow responses	Trim <code>SOUL.md</code> + <code>MEMORY.md</code> (less = faster)
SSH kills gateway	<code>systemctl --user restart openclaw-gateway</code>
Need diagnostics	<code>openclaw doctor --fix</code> fixes 90% of issues
Hermes Agent	
Gateway won't start	<code>hermes gateway run --replace</code> (kills old instance)
Auth expired	<code>hermes auth</code> (re-authenticate all providers)
Context too large	Send <code>/reset</code> to clear session
Need diagnostics	<code>hermes doctor --fix</code> fixes 90% of issues

Remote Access

```
ssh user@100.x.x.x #
Tailscale
tailscale serve 3000 #
Expose gateway
```

Keep Mac Awake

```
sudo pmset -a
disablesleep 1
caffeinate -d &
```

Git Sync Setup

```
gh repo create appie-
brain --private
cd ~/clawd
git clone
https://github.com/YOU/
appie-brain.git memory
```

Essential Links

A quick reference to the most important OpenClaw resources and community links.

Resource	URL
OpenClaw Docs	<u>docs.openclaw.ai</u>
OpenClaw GitHub	<u>github.com/openclaw/openclaw</u>
ClewHub Skills	<u>clawhub.com</u>
Community Discord	<u>discord.com/invite/clawd</u>
OpenClaw Subreddit	<u>r/openclaw</u>
Weblyfe AI	<u>weblyfe.ai</u>
Appie Kit	<u>github.com/S3YED/appie-kit</u>
Training Video	<u>youtube.com/watch?v=tnVVIFFVDQ8</u>
YouTube	<u>@weblyfenl</u>
LinkedIn	<u>Seyed Hosseini</u>

Your Next Steps

You have everything you need. Here's the exact order to get started: don't overthink it, just follow the steps.



Pick Your Hardware

Start with your existing Mac or PC. Don't buy anything new yet. You can always upgrade to a Mac Mini or VPS once you're hooked.



Sign Up for Essentials

GitHub (free), Telegram (free), Anthropic Claude Max (\$100-200/mo). That's all you need to start. Add other services as you need them.



Install & Onboard

Install Node.js, install OpenClaw, run `openclaw onboard`. Follow the wizard. It takes about 10 minutes.



Send Your First Message

Open Telegram, message your bot, send `/start`. When it responds, you have a working AI employee. Congratulations.



Iterate & Improve

Refine your SOUL.md. Install skills. Set up heartbeats. Clone the Appie Kit. Every week, your Appie gets better. That's the self-annealing loop in action.

Built by Appie-1 | [Weblyfe.ai](https://weblyfe.ai)

Version 4.5



Instagram

[@seyed.jpg](https://www.instagram.com/seyed.jpg)



YouTube

[@weblyfenl](https://www.youtube.com/@weblyfenl)



LinkedIn

[Seyed Hosseini](https://www.linkedin.com/in/Seyed Hosseini)



Website

weblyfe.ai



Email

hello@weblyfe.ai

Follow us for updates, tutorials, and community.

Green and gold. Built different. Let's go.

Build Your Own Appie v4.5, May 2026. This document was built with a Team of Appies.